

CST8507: NATURAL LANGUAGE PROCESSING

WEEK#9 **TRANSFORMER-BASED LANGUAGE MODELS**

DEVELOPED BY

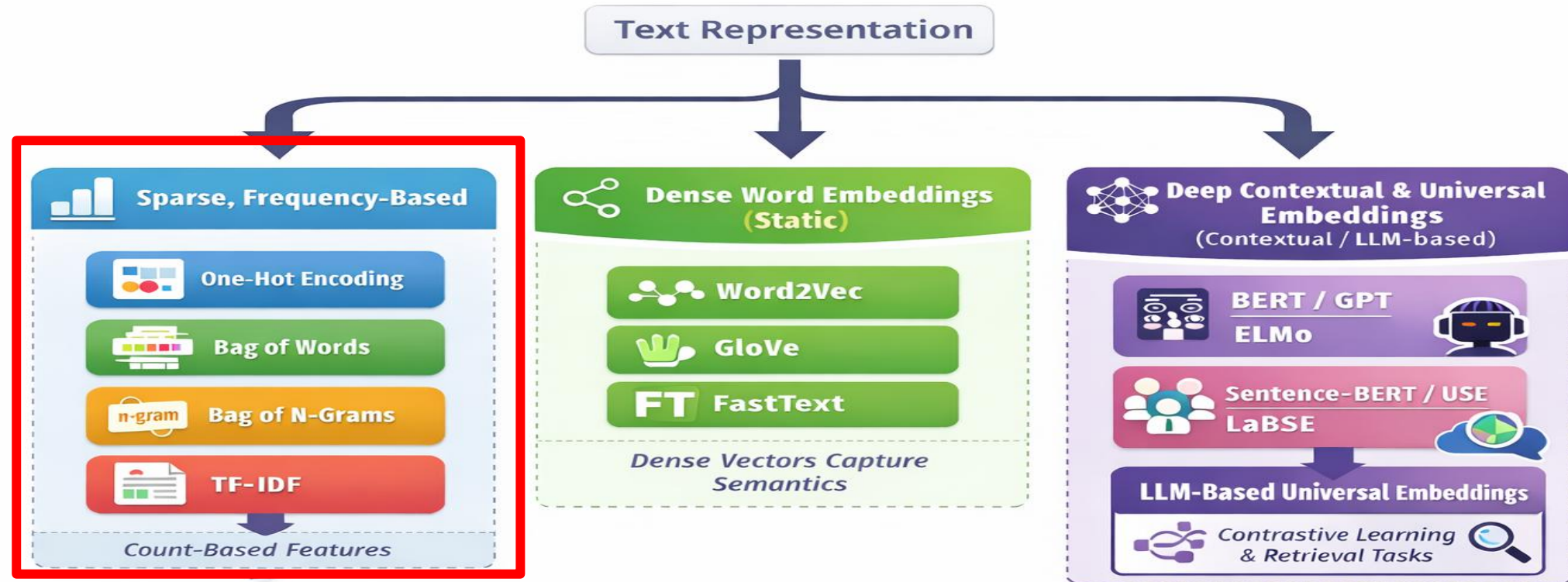
HALA OWN, PH.D.

Lesson Agenda

- ☐ Transformer Architecture
- ☐ Self-Attention mechanisms
- ☐ Transfer Learning and Fine-Tuning
- ☐ Applications of transformer-based language models
- ☐ Drawbacks and variants of Transformers



Text Representation Techniques



Problem with static embeddings (word2vec)

- Fixed embeddings: One word = one vector (no context)
- Fixed at Training Time
- Out-of-vocabulary (OOV) problem
- Morphological Blindness(run, running, runner)



Contextual Embeddings

- Representation of meaning of a word should be different in **different contexts!**
- Each word has a **different vector**
- The meanings depend on the **surrounding words**



Self attention: Motivations

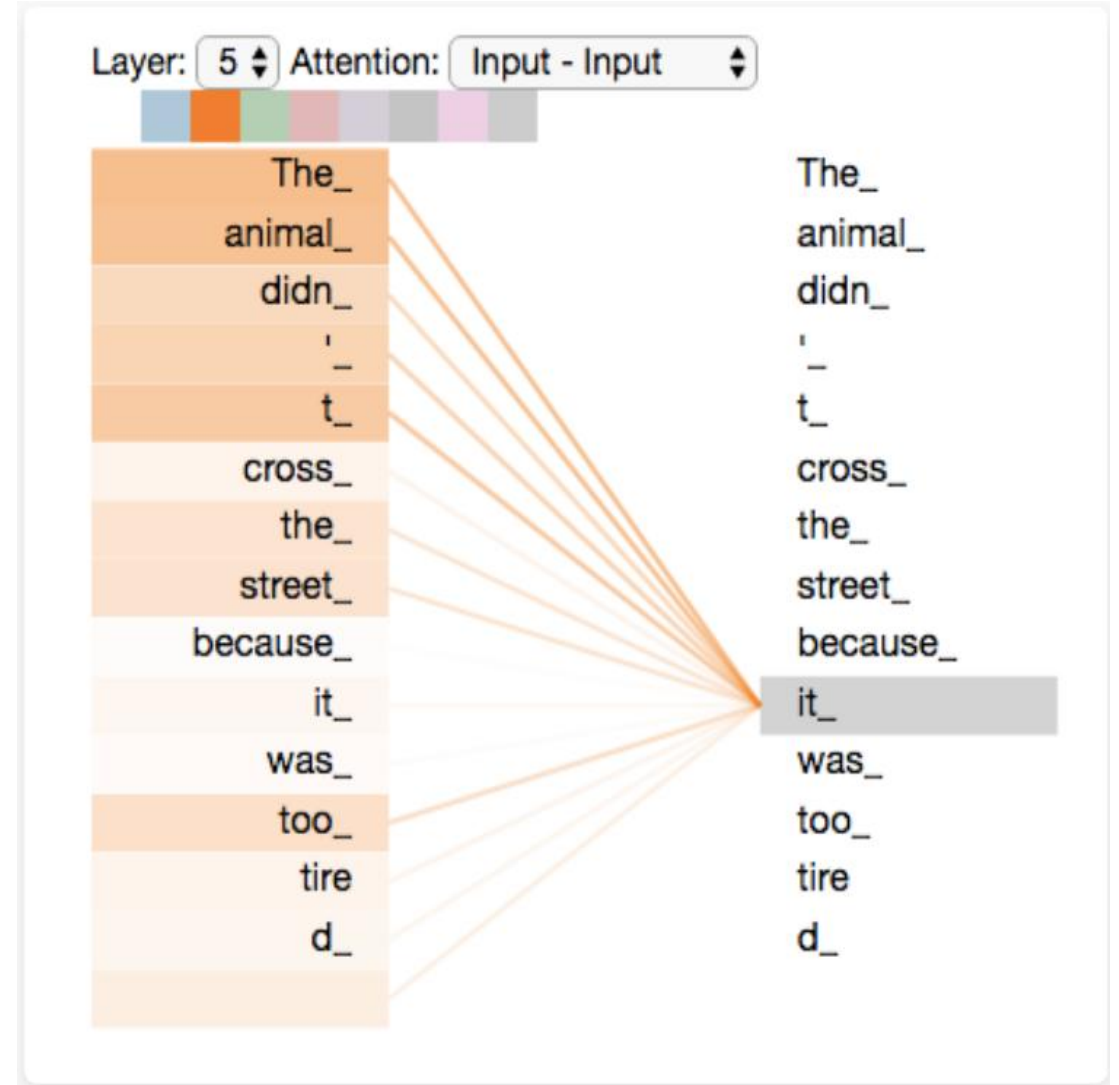
- Build up the **contextual embedding** for a word by selectively integrating information from all **neighboring words** , not equally but **weighted by relevance**.
- Each **word evaluates the importance** of the other words in the sentence and focuses more on those that provide useful context, while giving **less weight to less relevant** words.



What is Self-attention

Every word in a sequence asks: "***Which other words in this sentence are most relevant to understanding me?***"

Self-attention is the mechanism that answers that question for every word, simultaneously.



Why Multi-Head Attention?

- The Problem with Single-Head Attention
 - one weighted blend of all words.

The animal didn't cross the street because it was too tired.

Coreference



it → animal

Syntactic

it
→ subject of "was"

It was too tired.

Semantic



it → living thing

The word "it" needs to resolve three things:

- Refers to the **animal**
- Acts as the **subject**
- Must mean something **alive**.

Transformers(2017)



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

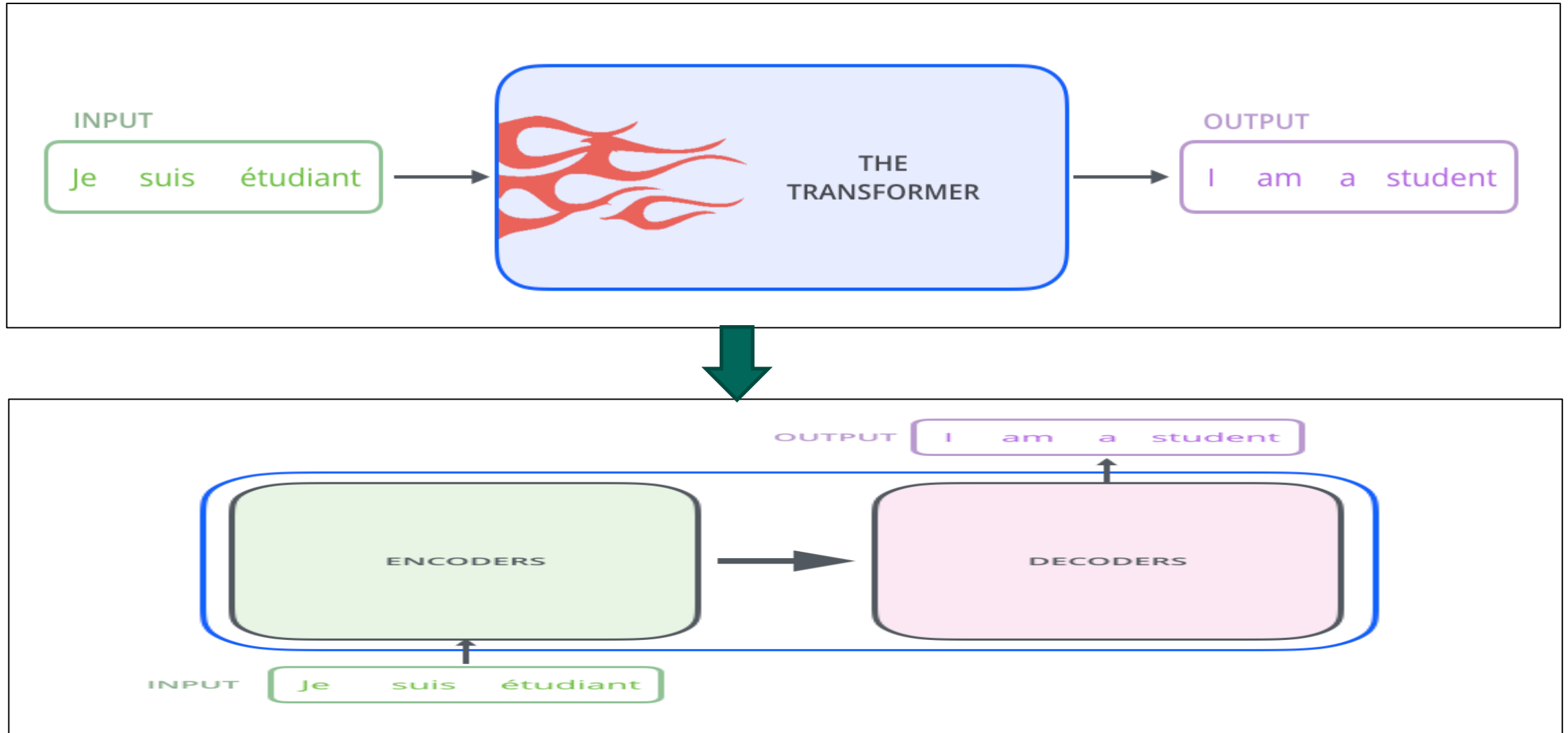
*<https://arxiv.org/abs/1706.03762>

What is Transformer

- The Transformer in NLP is a novel architecture that aims to **solve sequence-to-sequence** tasks while handling long-range dependencies with ease.
- The Transformer was proposed in the paper *Attention Is All You Need* ^{*}.
- Relying entirely on **self-attention** to compute representations of its input and output.

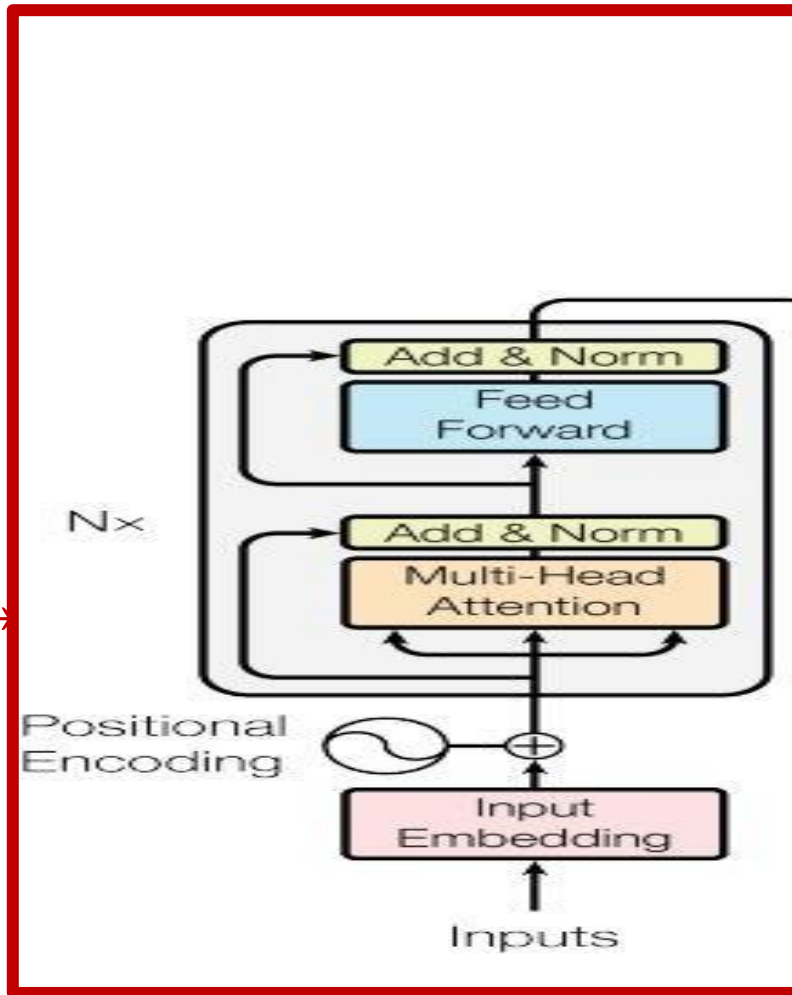
^{*}<https://arxiv.org/abs/1706.03762>

Transformer Architecture

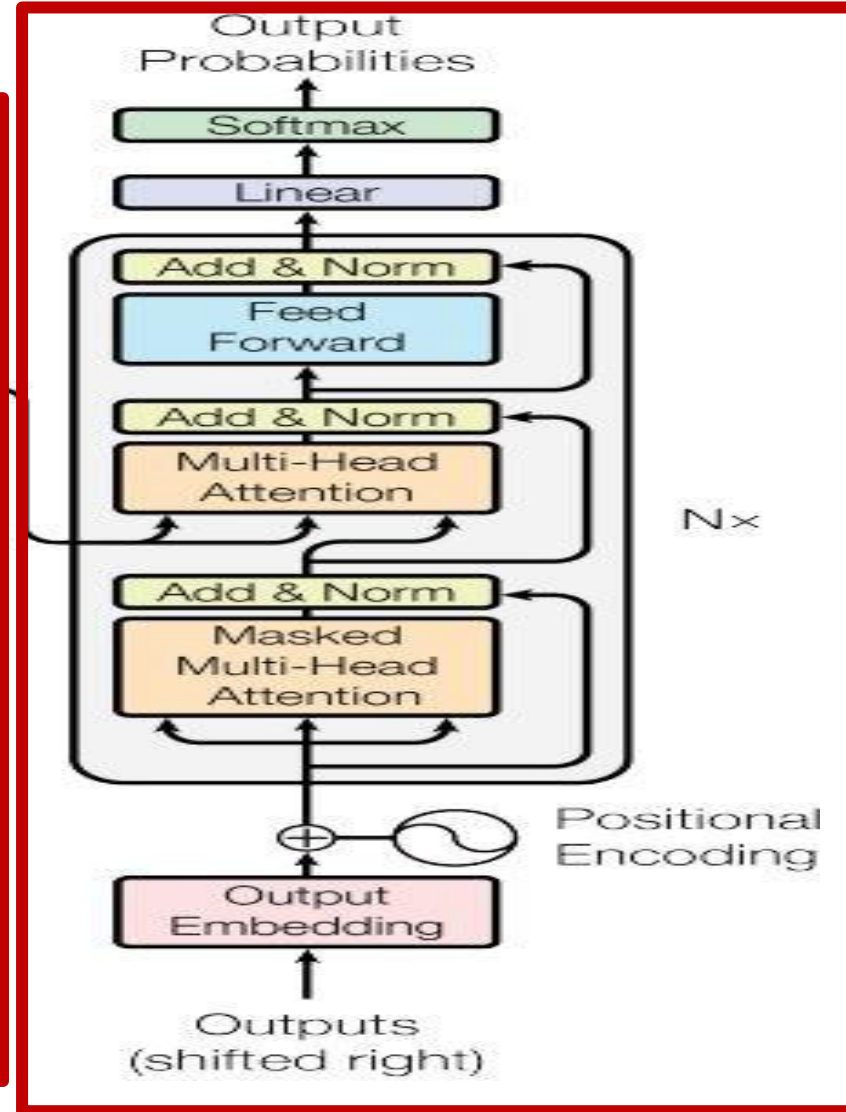


Transformer's Model Architecture

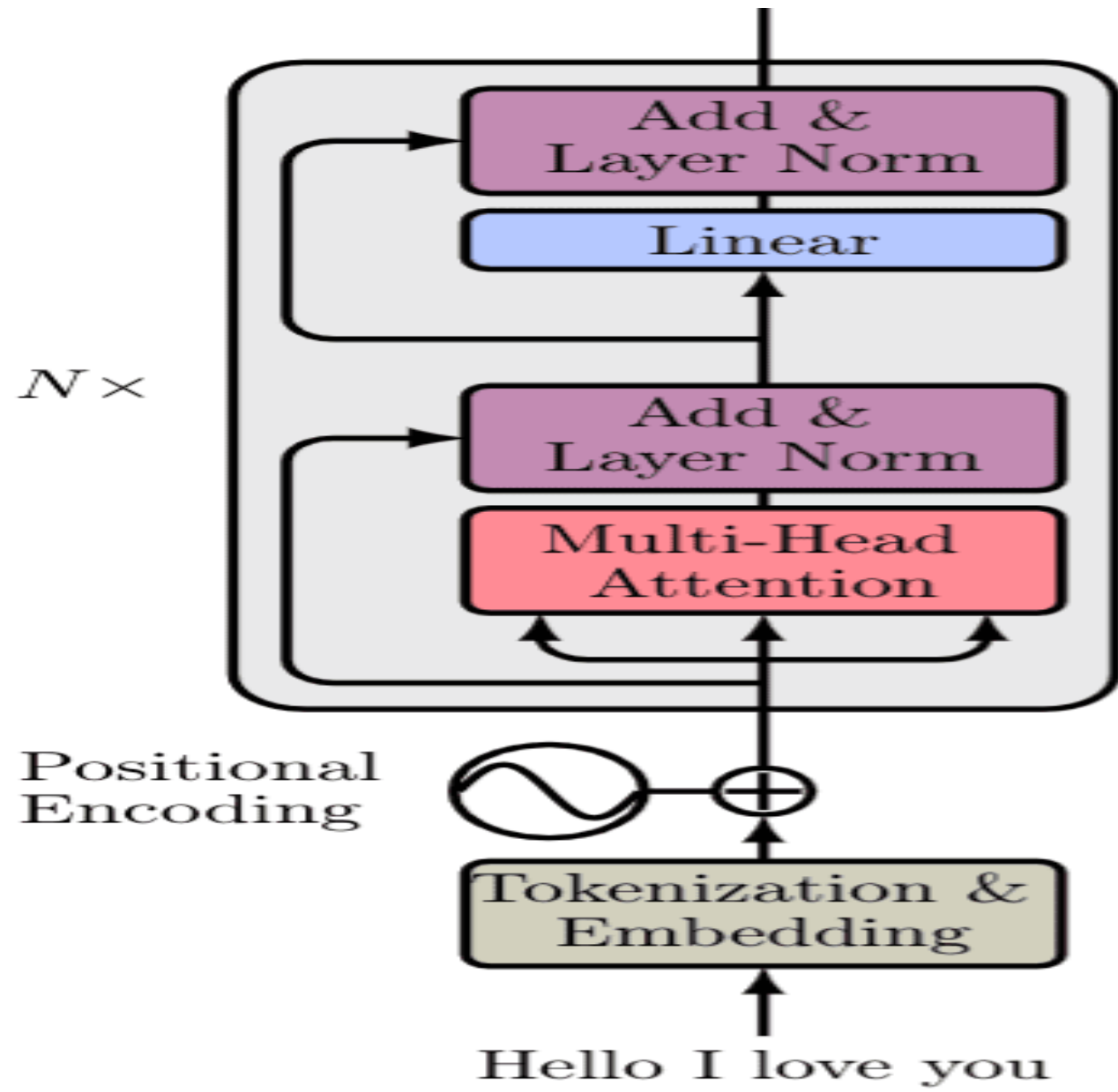
Encoder →



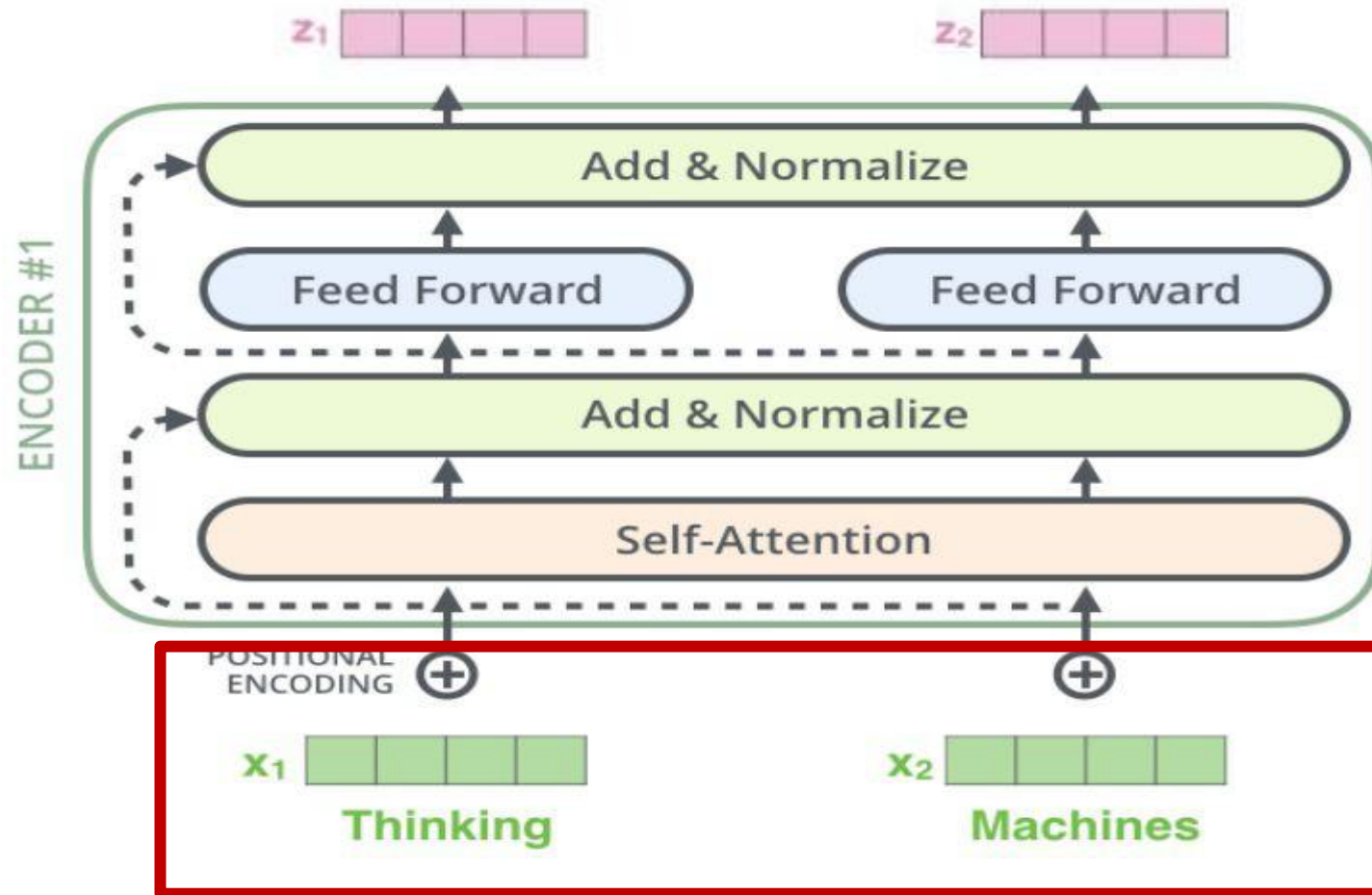
← Decoder



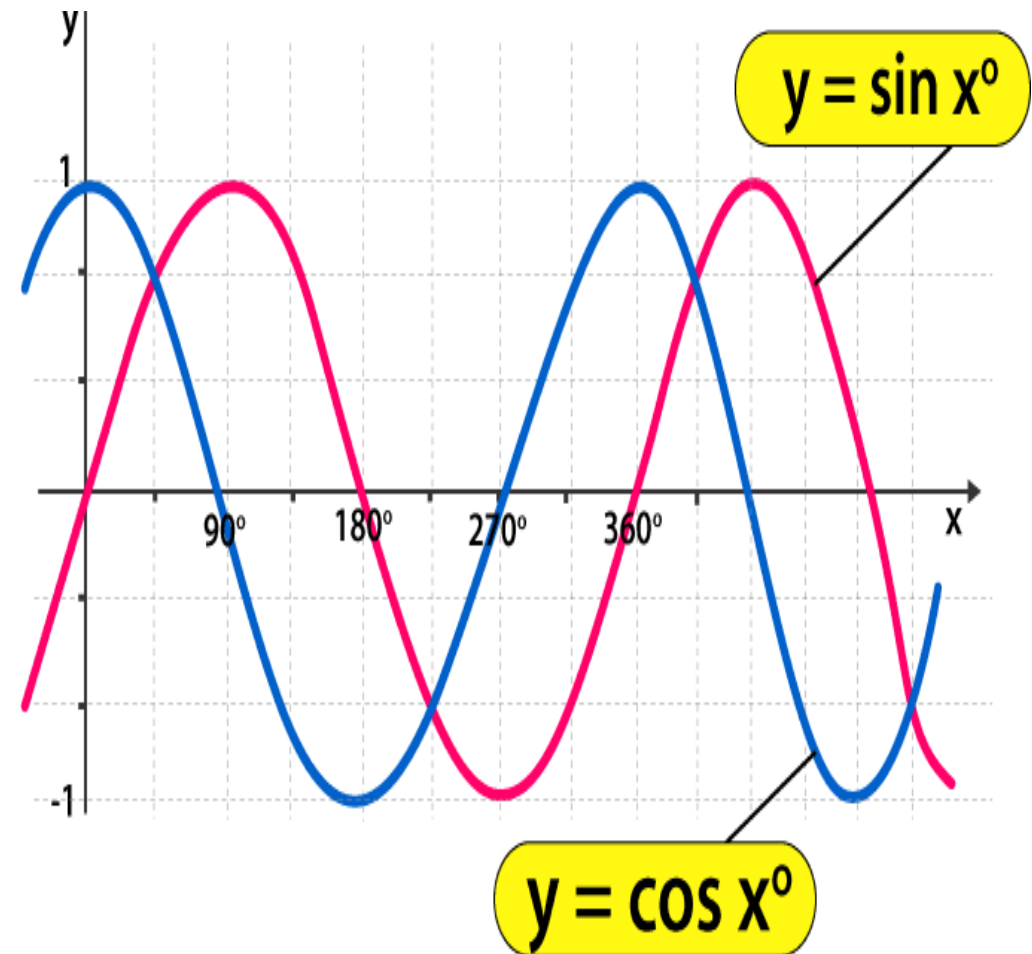
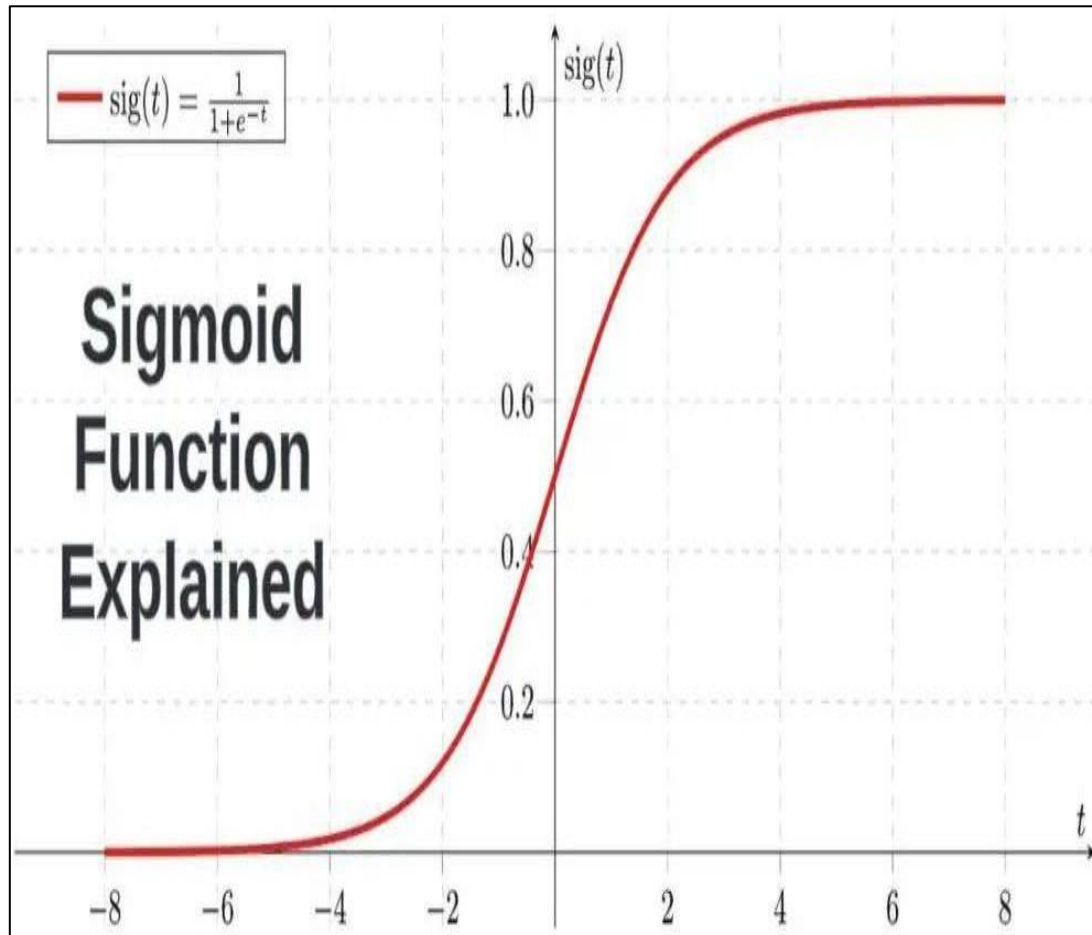
Encoder Anatomy



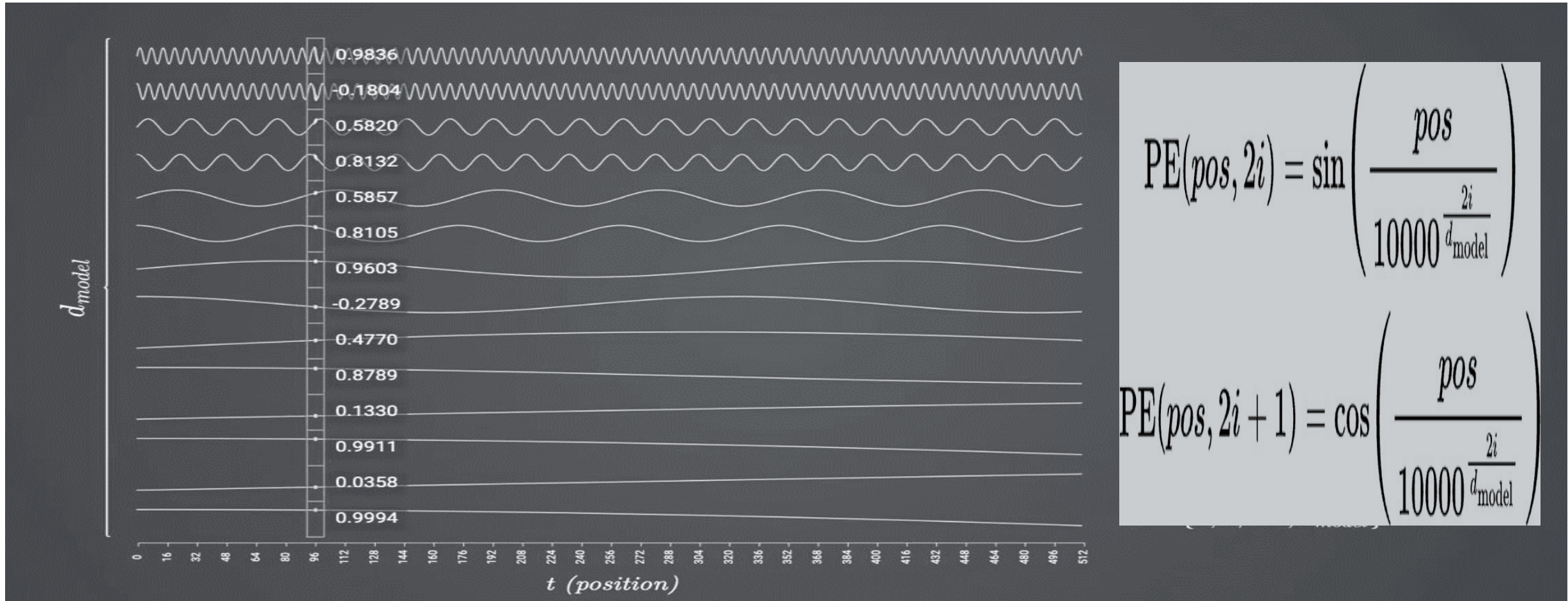
Encoder: Positional Encoding



Encoder: Positional Encoding

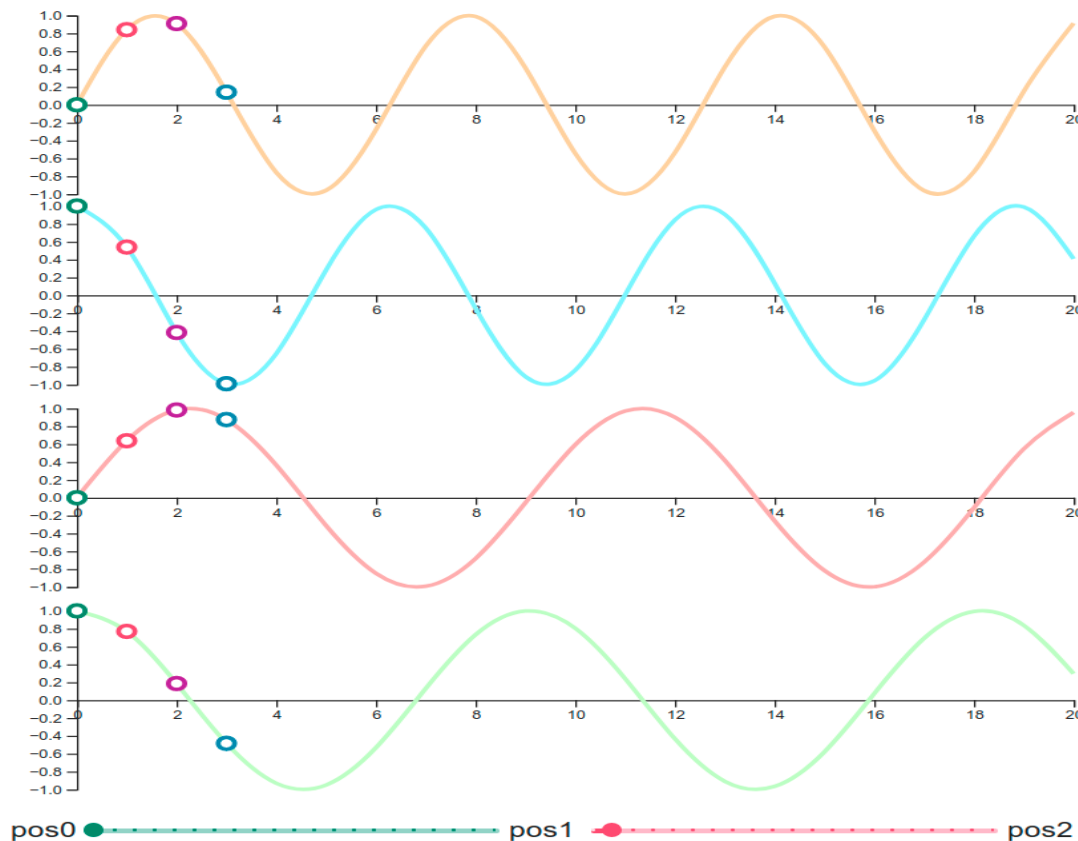


Encoder: Positional Encoding...



Encoder: Positional Encoding

The positional vector is added to the embedding vector of the word at **position 2** in the input sequence.



p0	p1	p2	p3	
0.000	0.841	0.909	0.141	i=0
1.000	0.540	-0.416	-0.990	i=1
0.000	0.638	0.983	0.875	i=2
1.000	0.770	0.186	-0.484	i=3

Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Settings: $d = 50$

The value of each positional encoding depends on the *position* (pos) and *dimension* (d). We calculate result for every *index* (i) to get the whole vector.

Positional Encoding :Example

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

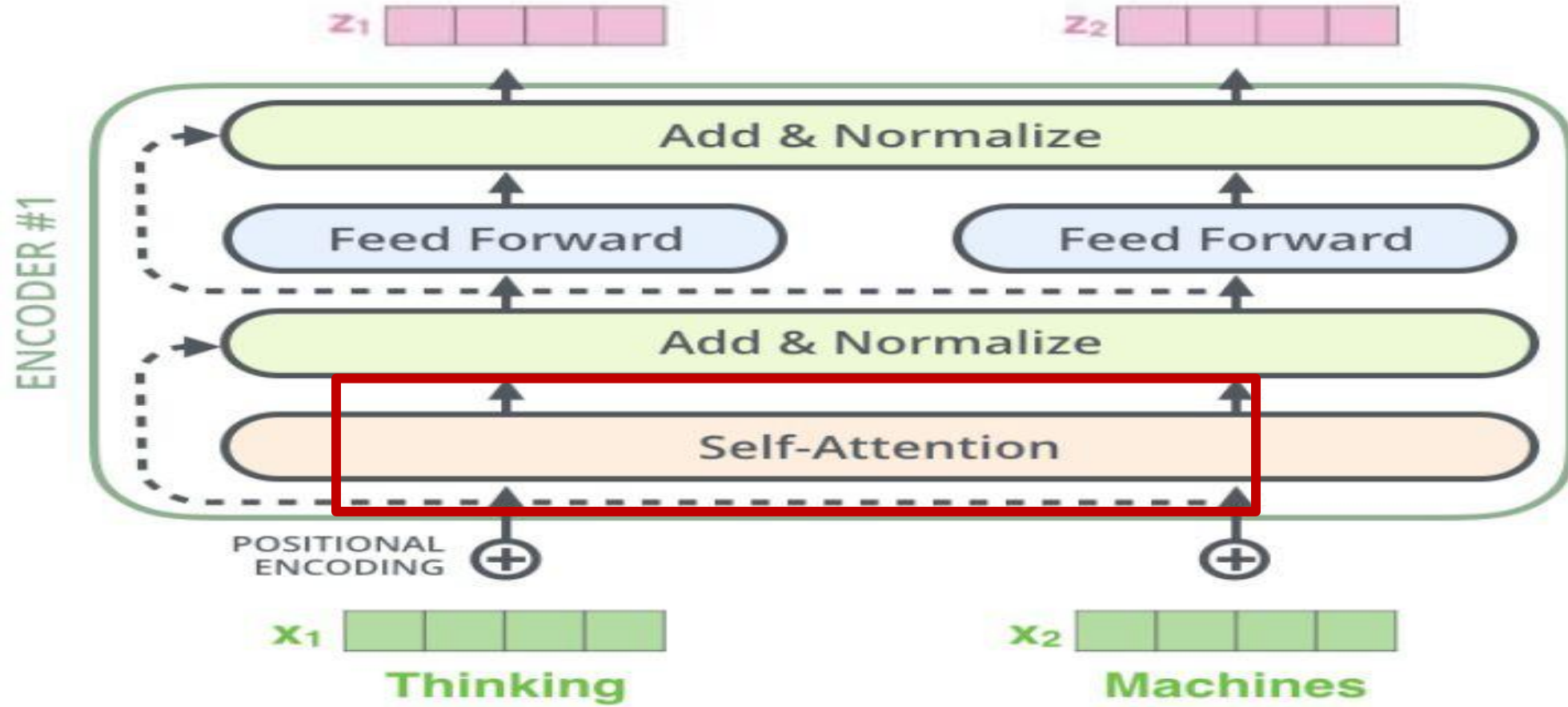
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

“I love NLP” where d=4

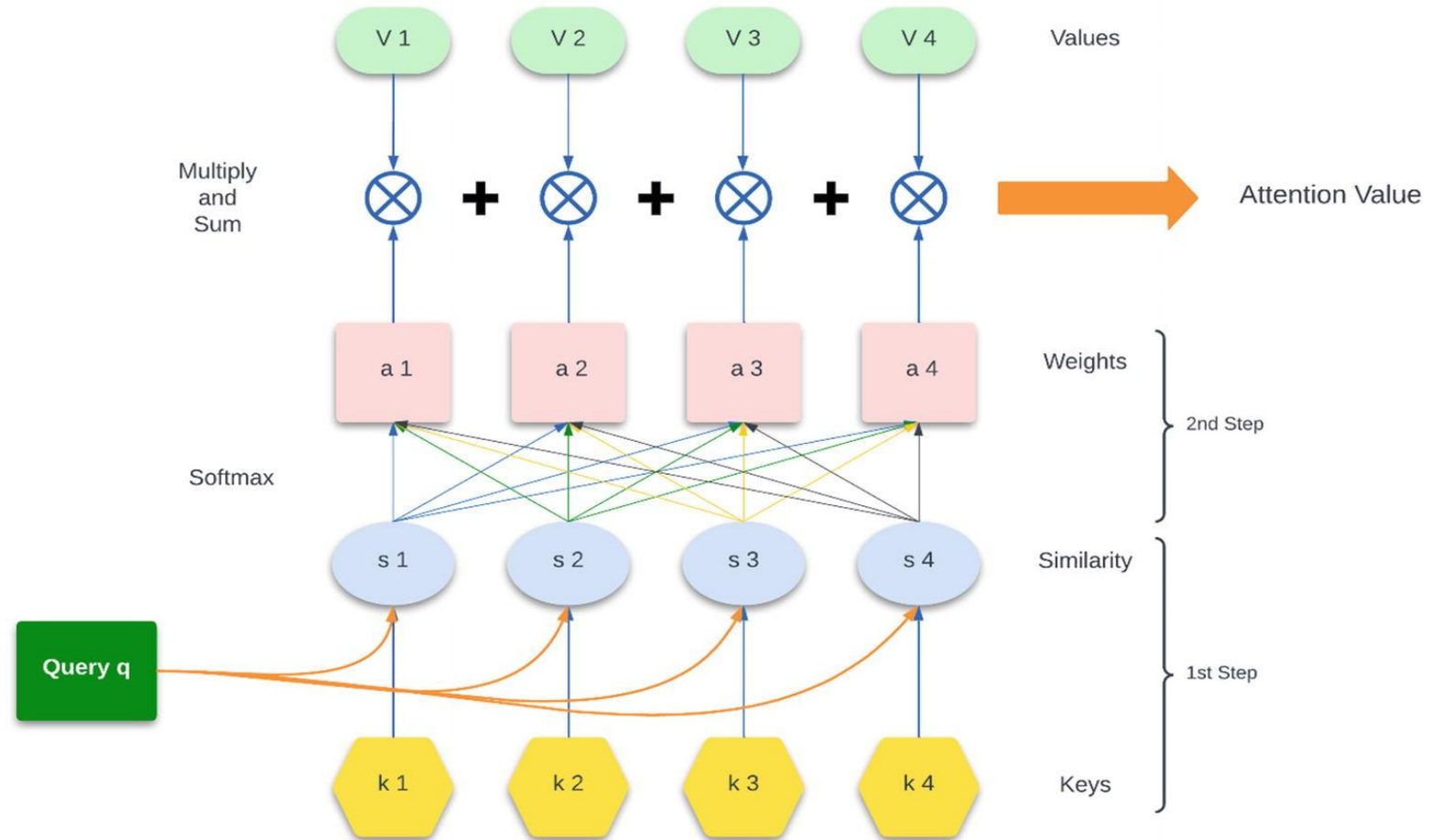
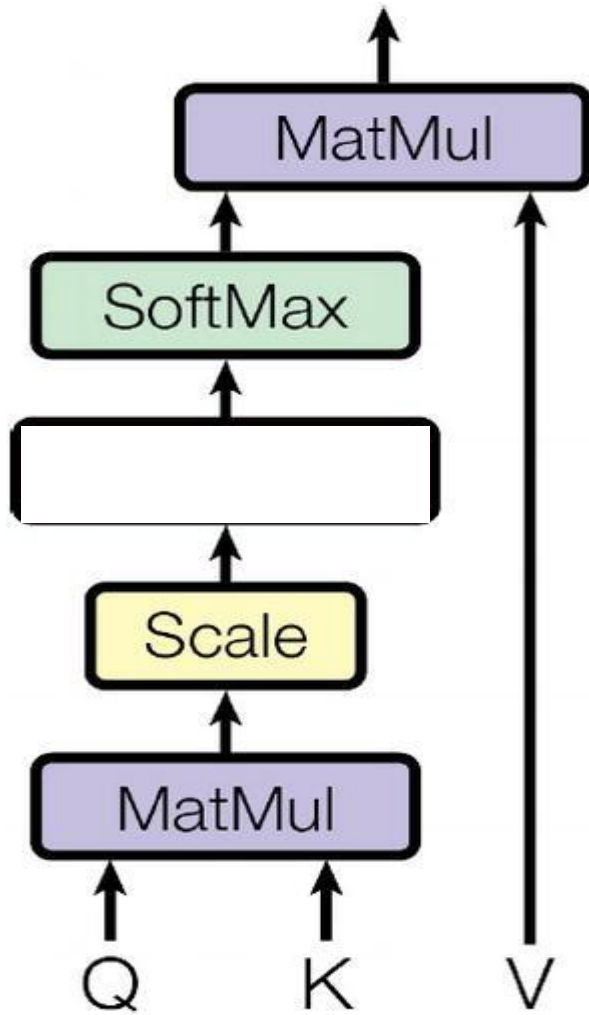
Word	Position	Calculation	Encoding vector
"I"	0	$PE(0, 0) = \sin(0) = 0.0000$ $PE(0, 1) = \cos(0) = 1.0000$ $PE(0, 2) = \sin(0 / 100) = \sin(0) = 0.0000$ $PE(0, 3) = \cos(0 / 100) = \cos(0) = 1.0000$	[0, 1, 0, 1]
"love"	1	$PE(1, 0) = \sin(1) \approx 0.8415$ $PE(1, 1) = \cos(1) \approx 0.5403$ $PE(1, 2) = \sin(1 / 100) = \sin(0.01) \approx 0.0100$ $PE(1, 3) = \cos(1 / 100) = \cos(0.01) \approx 1.0000$	[0.8415, 0.5403, 0.01, 0.9999]
"NLP"	2	$PE(2, 0) = \sin(2) \approx 0.9093$ $PE(2, 1) = \cos(2) \approx -0.4161$ $PE(2, 2) = \sin(2 / 100) = \sin(0.02) \approx 0.0200$ $PE(2, 3) = \cos(2 / 100) = \cos(0.02) \approx 1.0000$	[0.9093, -0.4161, 0.02, 0.9998]



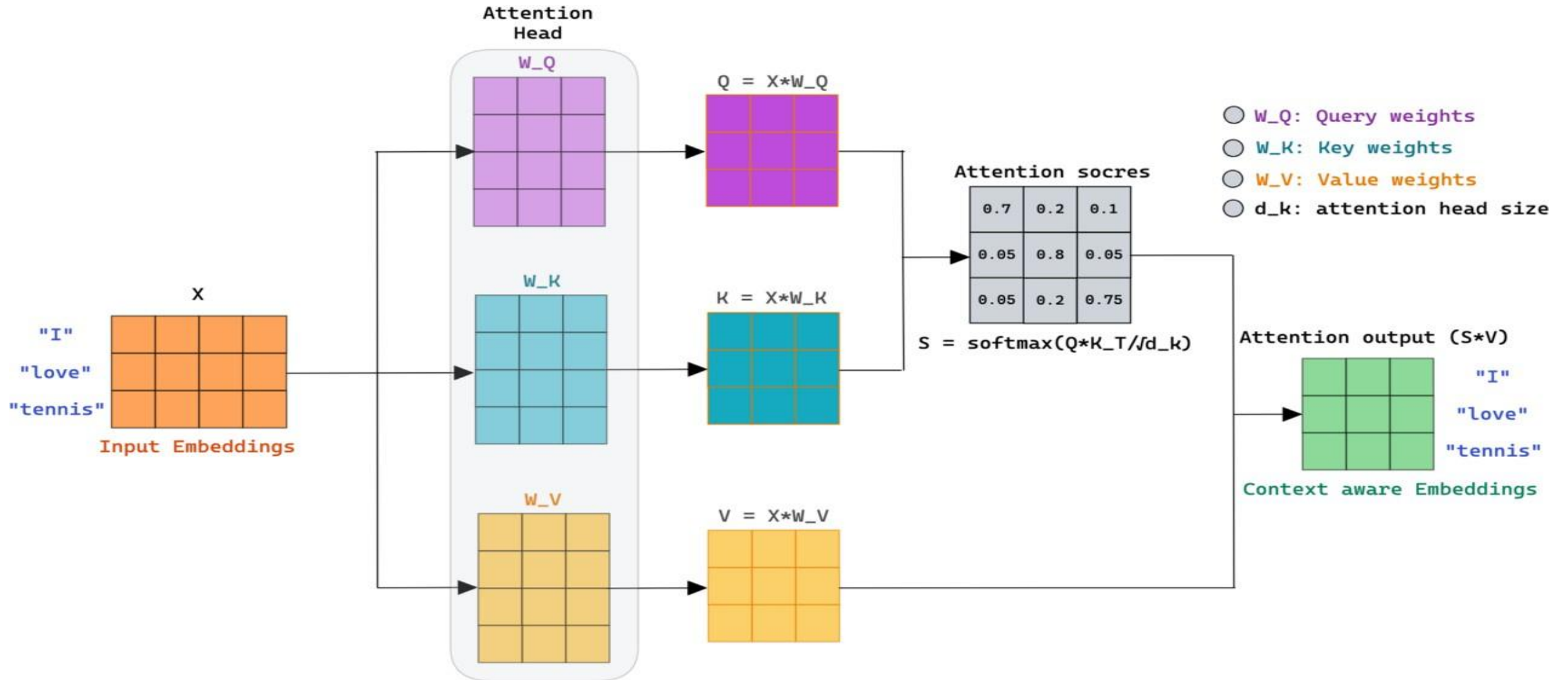
Encoder: Self Attention



The Attention Mechanism Workflow

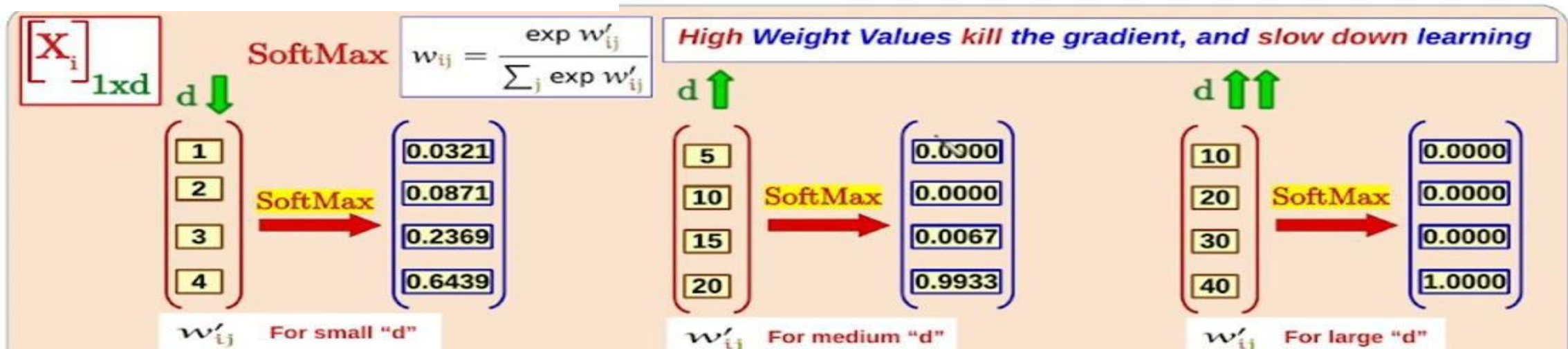


Self-attention



Scale Dot-Product Motivation: SoftMax Sensitivity

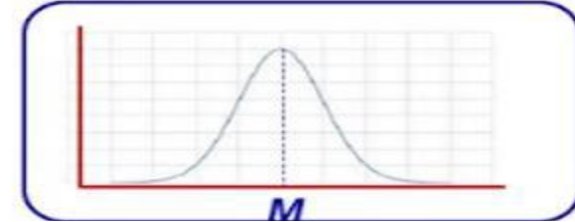
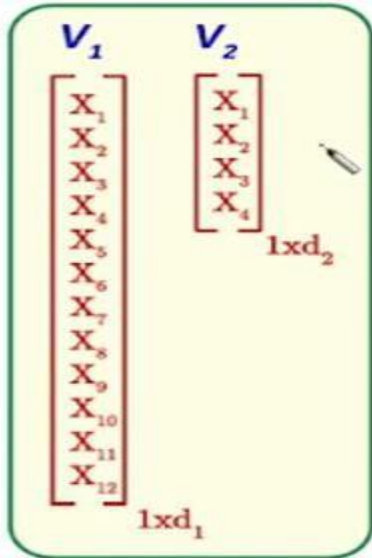
Small changes in logits can drastically change probabilities due to exponentiation.



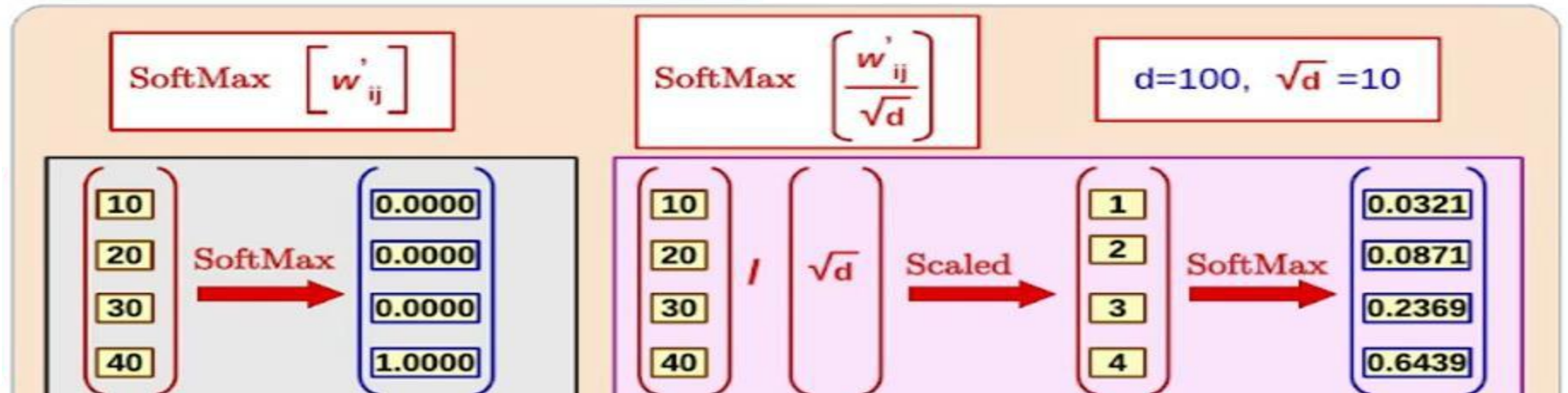
Scale Dot-Product Motivation: SoftMax Sensitivity...

Self Attention Mechanism

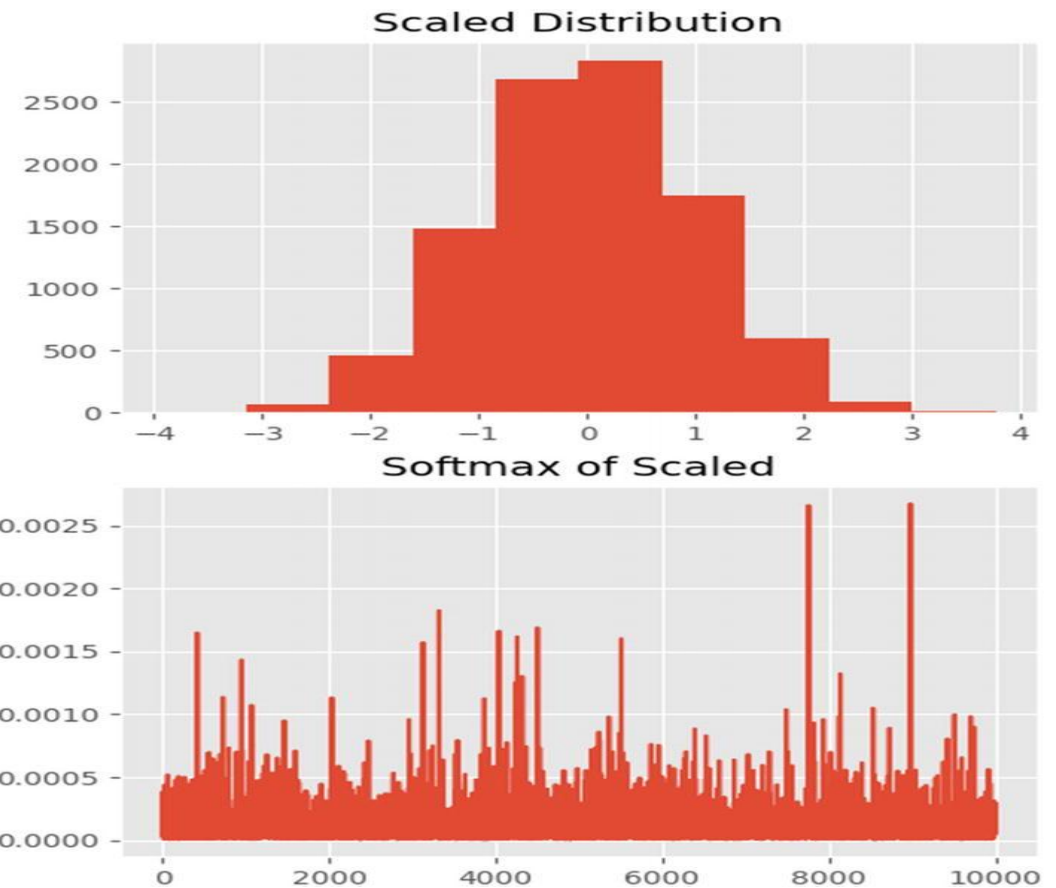
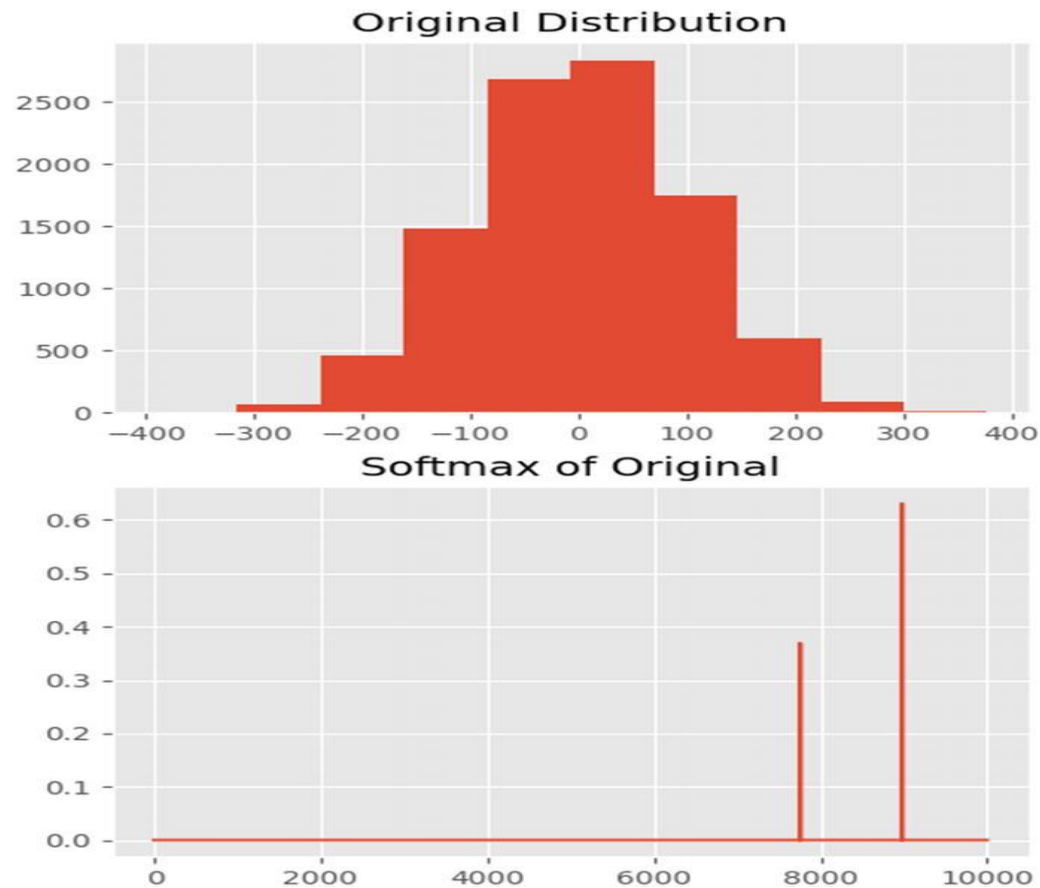
(Scaling based on Square Root of Embedding Vector Dimension)



$\sqrt{d}$ is a good choice for scaling

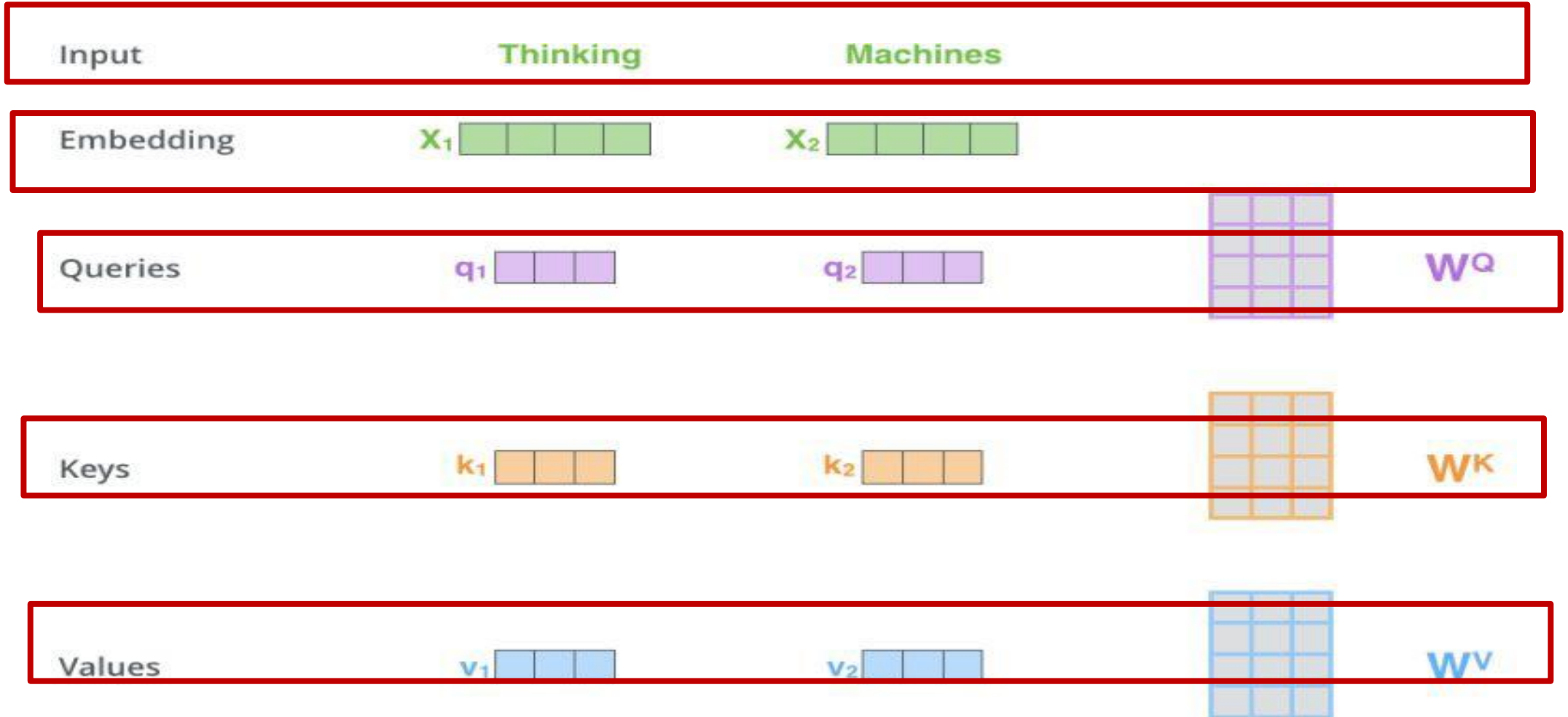


Scale Dot-Product: why?



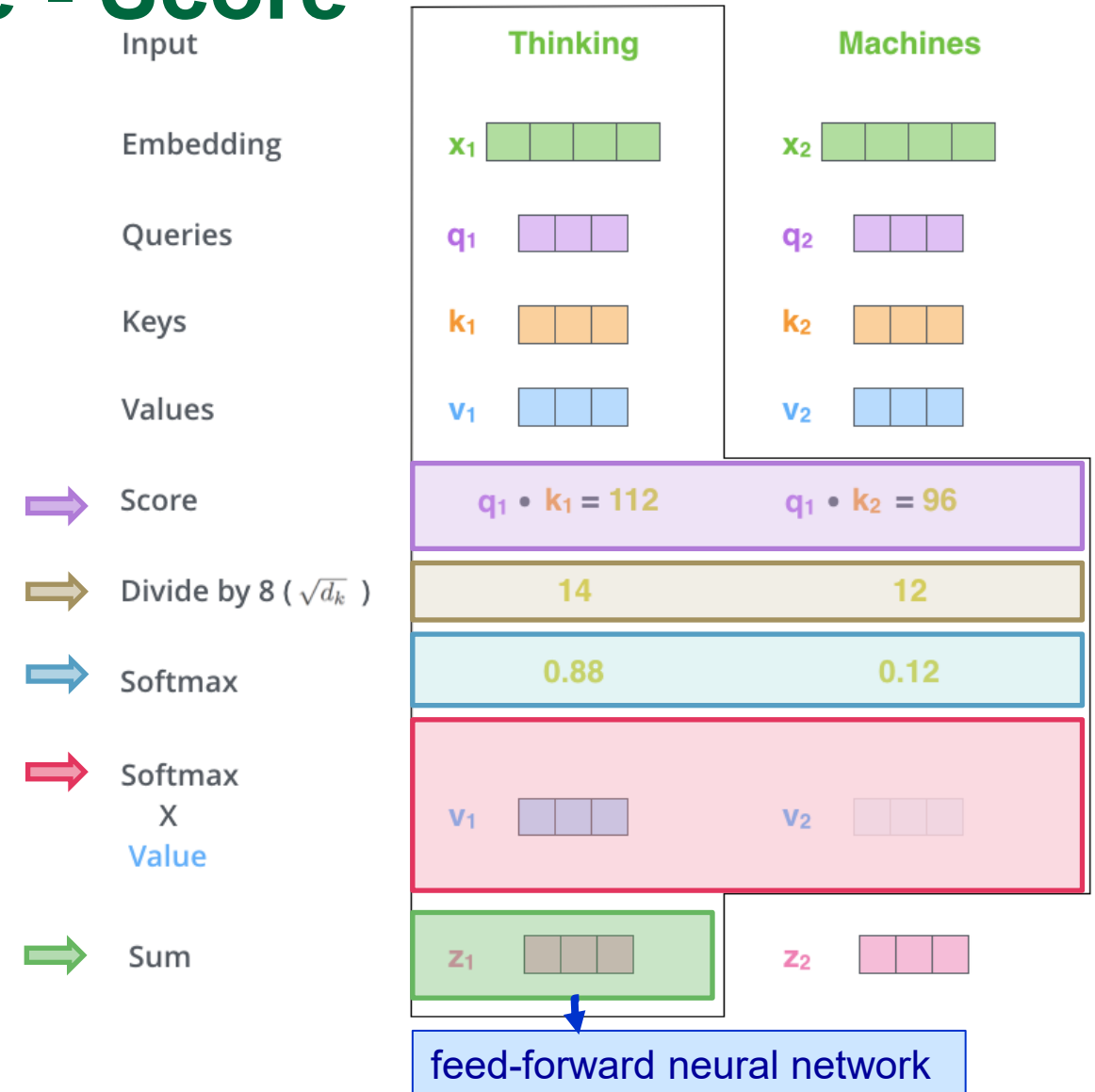
Encoder :Self (Scale Dot-Product) Attention

D=64

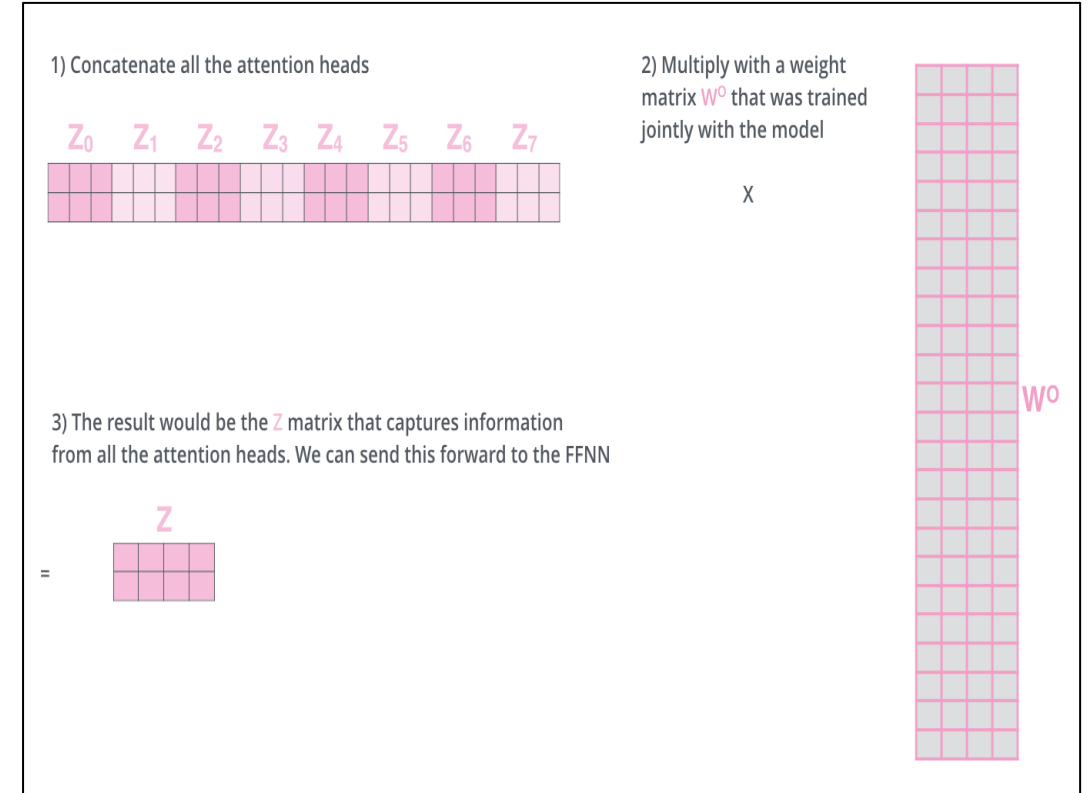
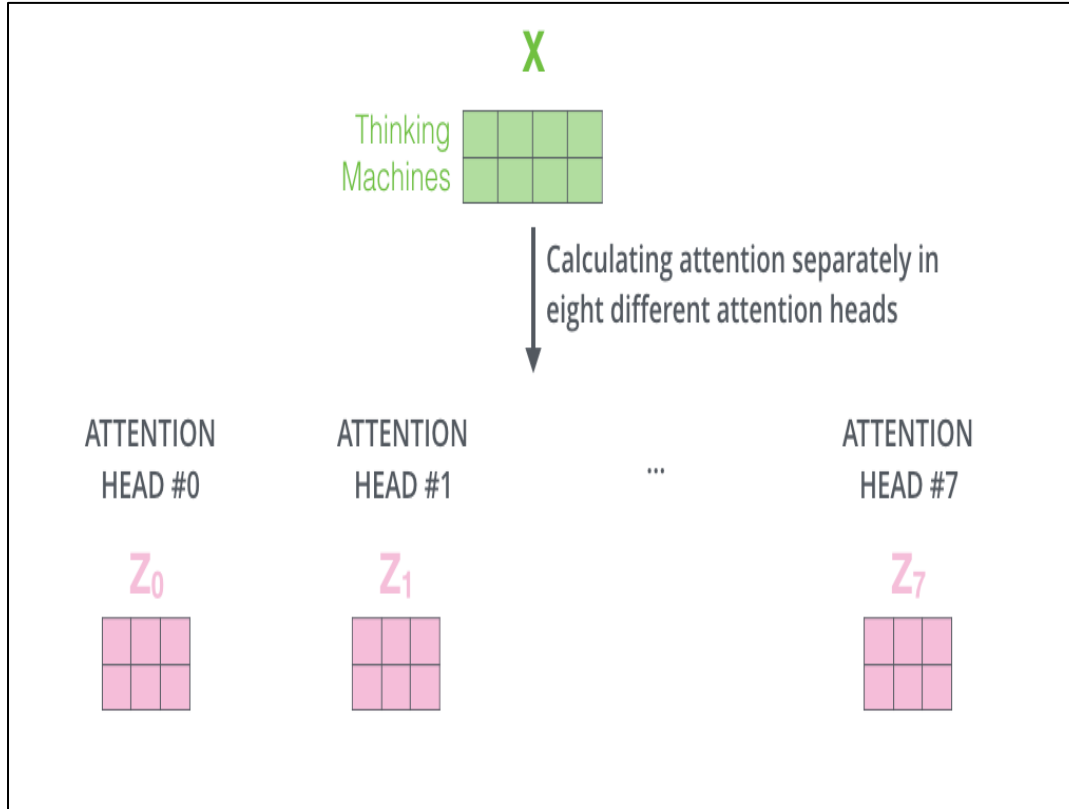


Transformer Architecture - Score

- Calculate scores: the dot product of the query vector with the key vector of the respective word.
- Divide the scores by $\sqrt{64}$
- Calculate Softmax
- Multiply each value vector by the softmax score
- Sum up the weighted value vectors.



Encoder :Multi-headed” Attention

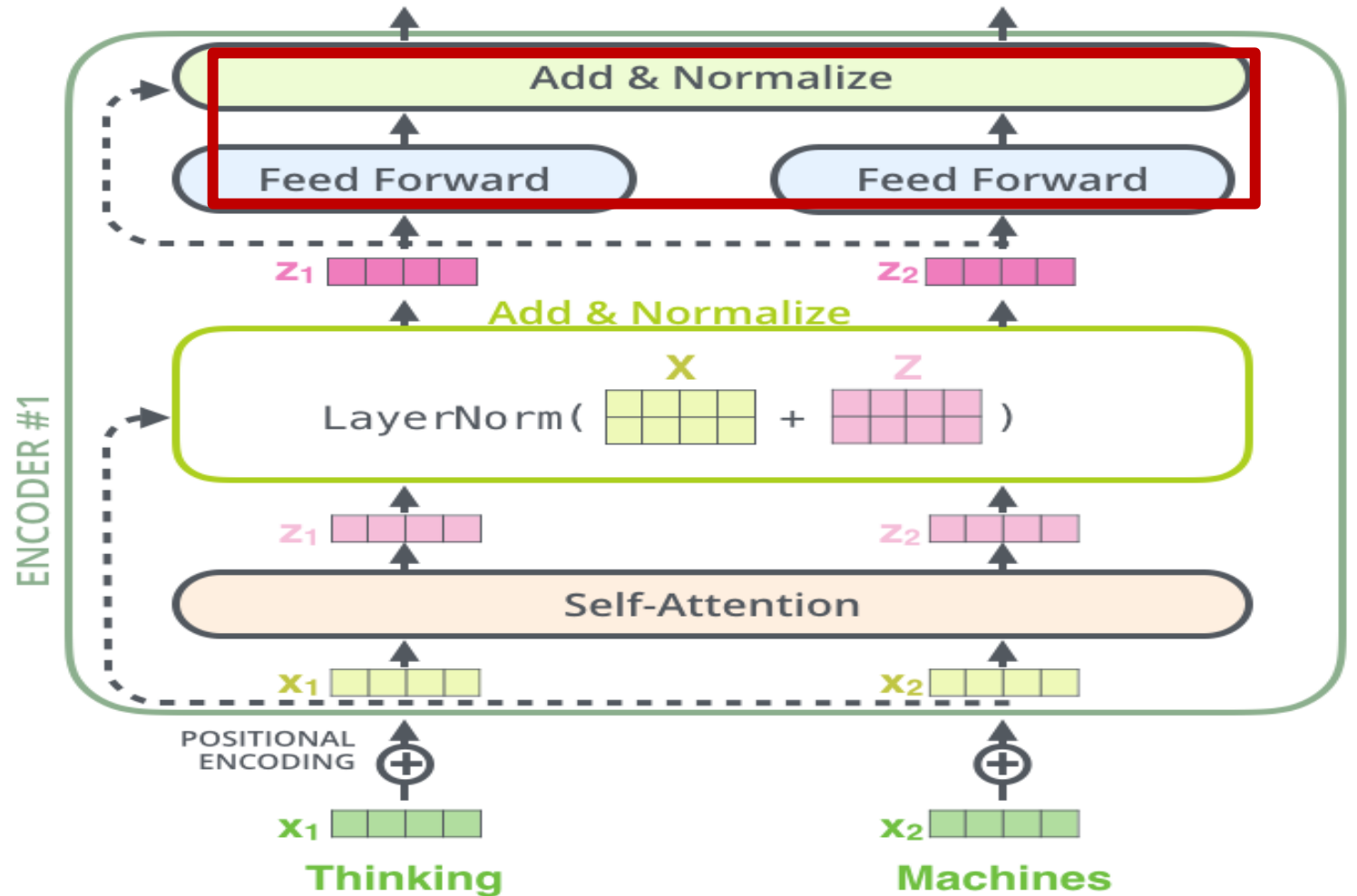


The Feed-Forward Layer(position-wise feed-forward layer)

$$\text{FFN}(Z) = \text{ReLU}(Z \cdot W_1 + b_1) \cdot W_2 + b_2$$

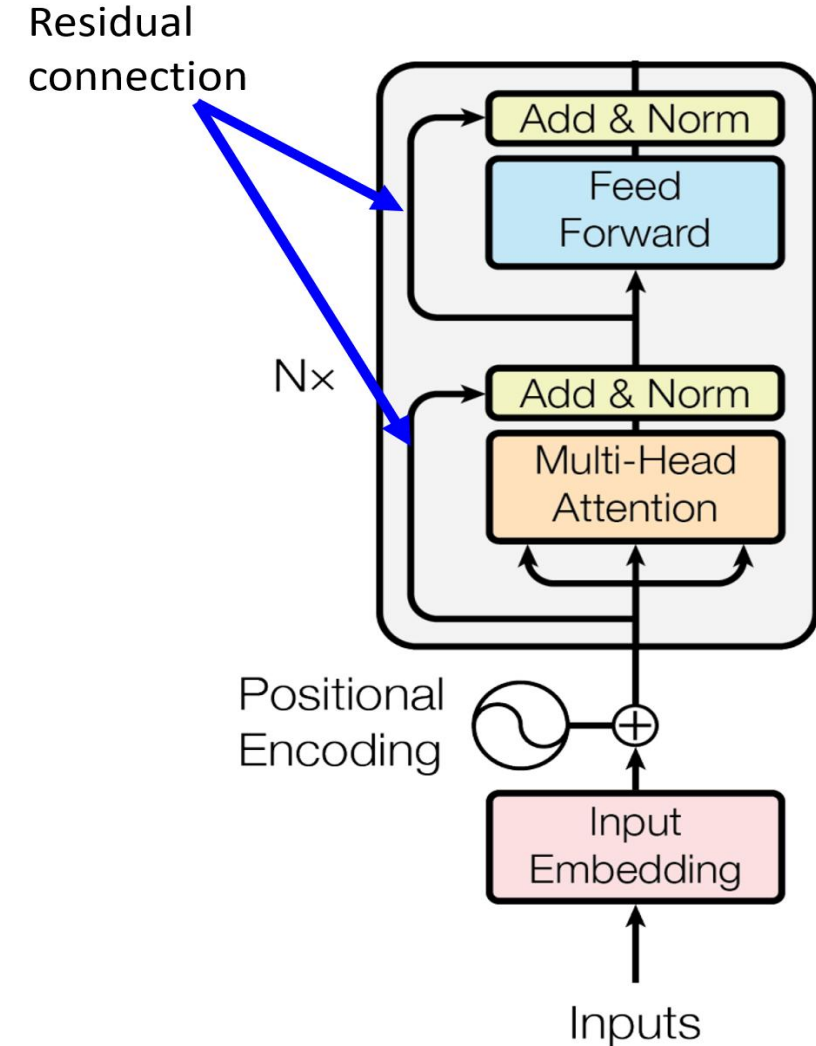
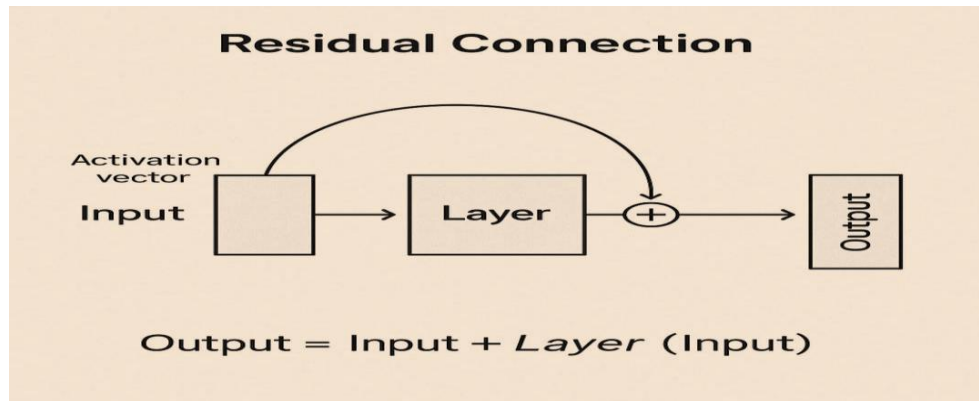
W_1, W_2 : weight matrices

b_1, b_2 : bias vectors



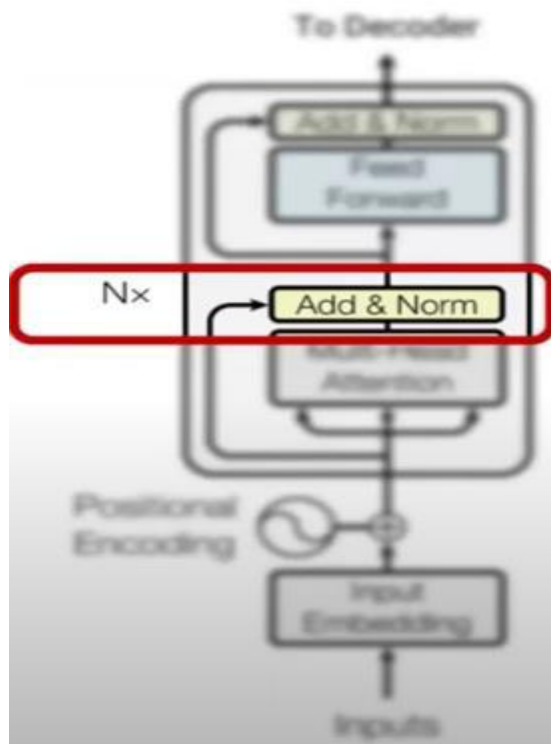
Residual Connections

- let each layer add refinements to the input rather than replace it

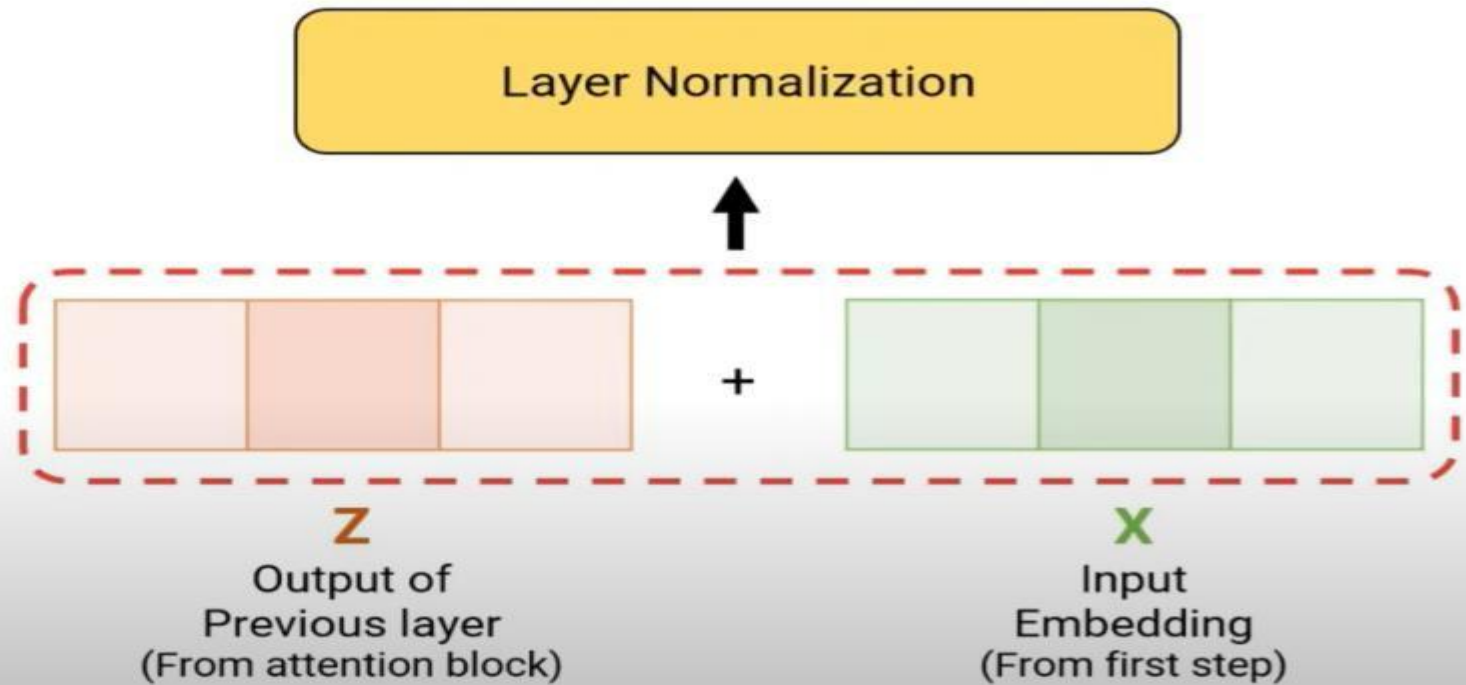


Add & Norm

Encoder



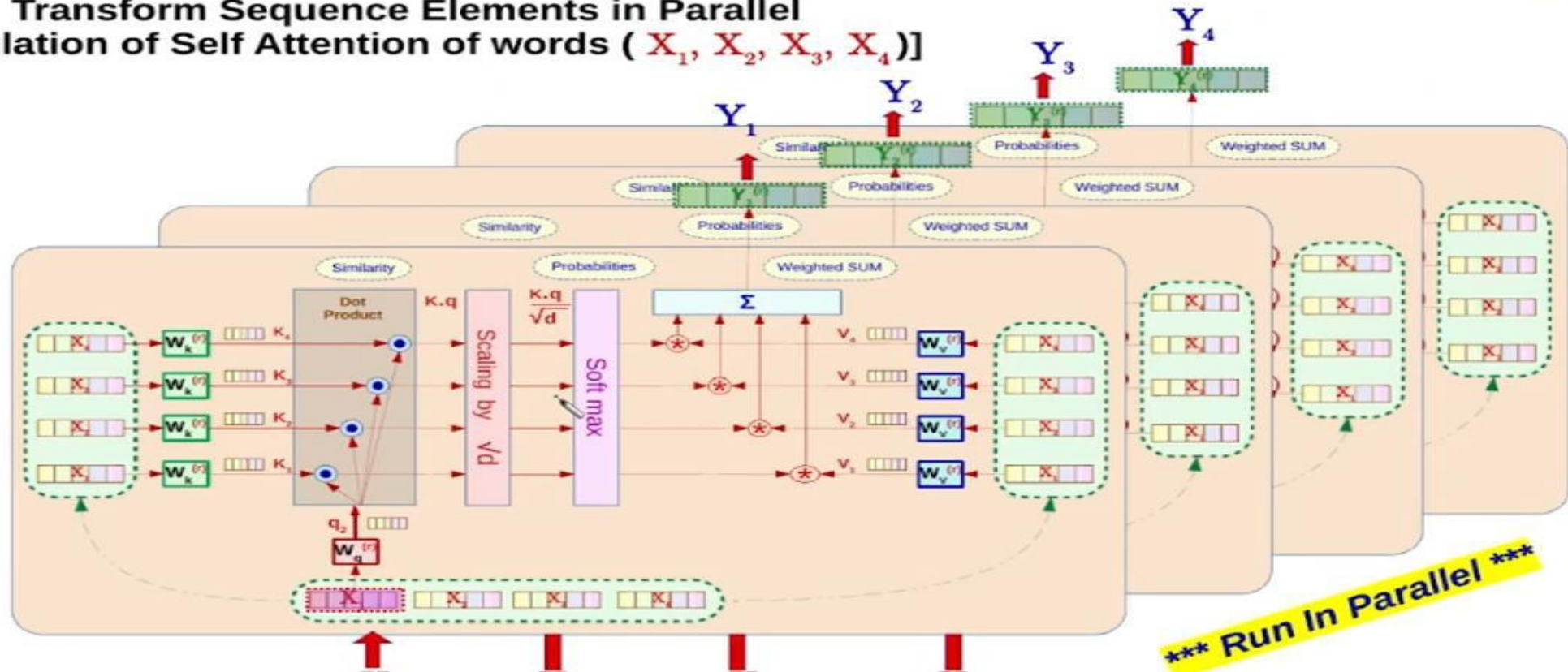
Add and Normalization



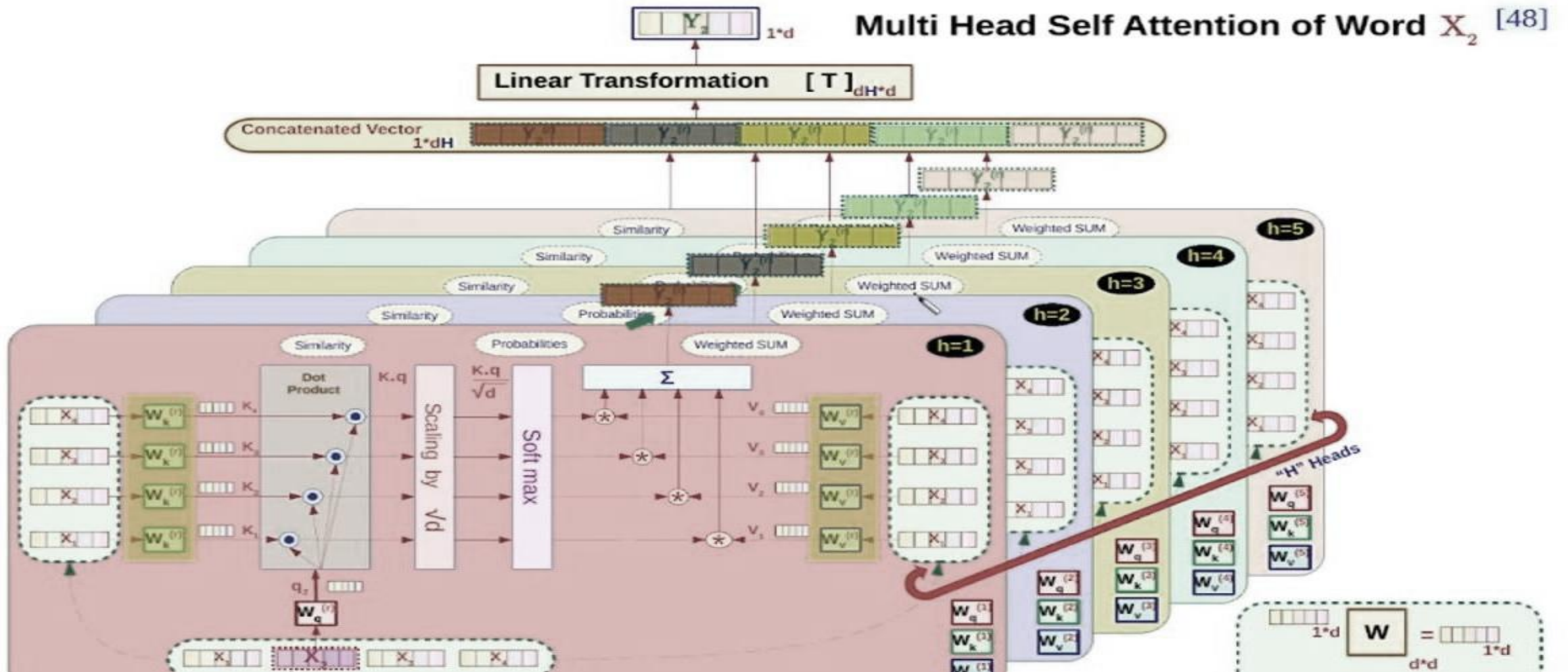
Parallel processing

Parallel Processing
Transform Sequence Elements in Parallel
[Calculation of Self Attention of words (X_1, X_2, X_3, X_4)]

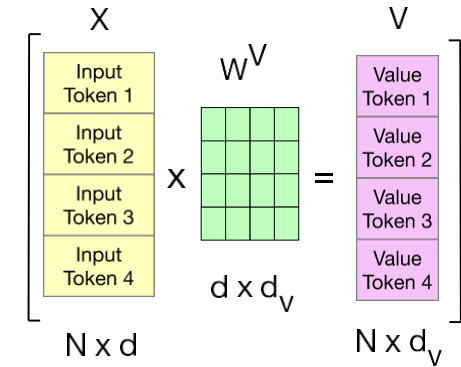
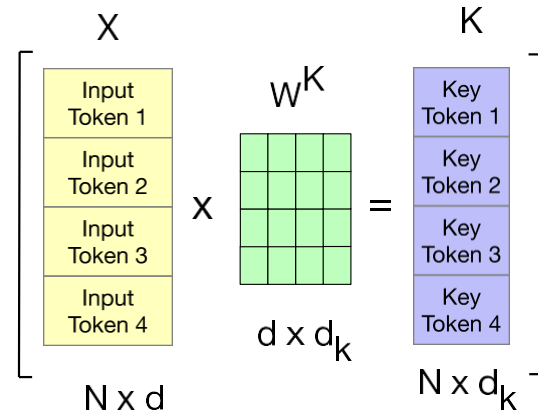
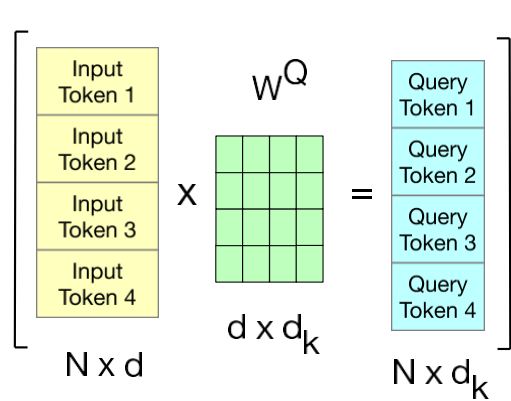
[33]



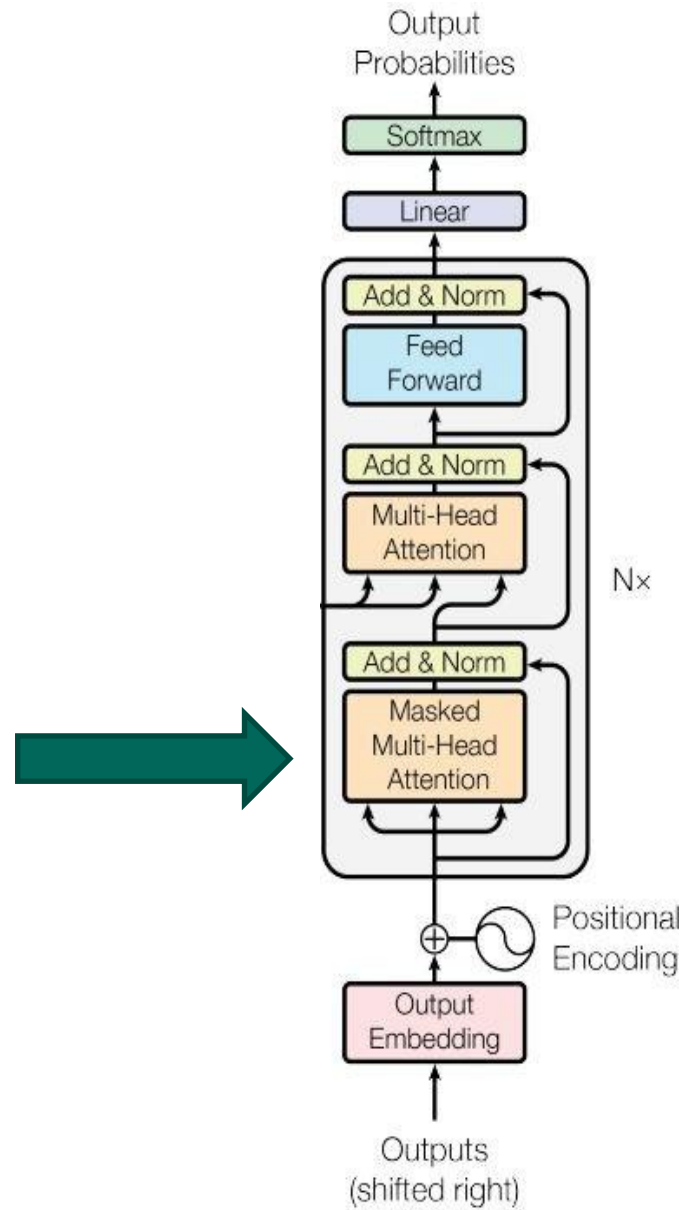
Parallel processing...



Parallel Computation of Query, Key, and Value in Self-Attention



Decoder



← Decoder



Masking the future in self-attention

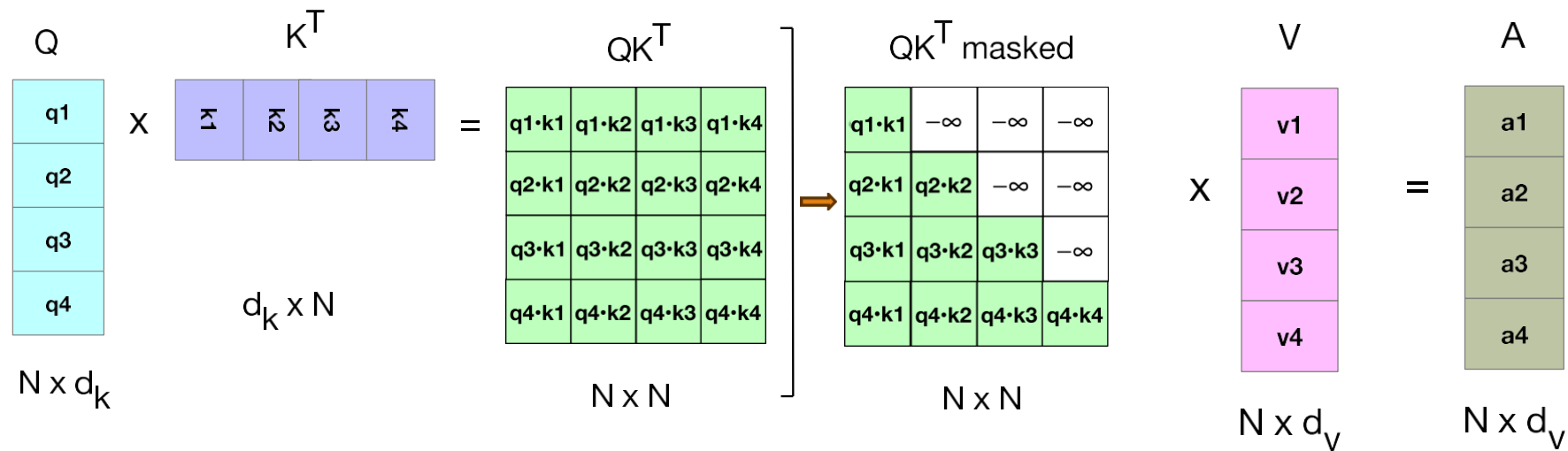
We can look at these
(not greyed out) words

mask out attention to future words by setting
attention scores to $-\infty$.

For encoding
these words

	[START]	I	love	NLP
[START]		$-\infty$	$-\infty$	$-\infty$
I			$-\infty$	$-\infty$
love				$-\infty$
NLP				

Masking the future in self-attention...



Cross-Attention: The Bridge Between Encoder and Decoder

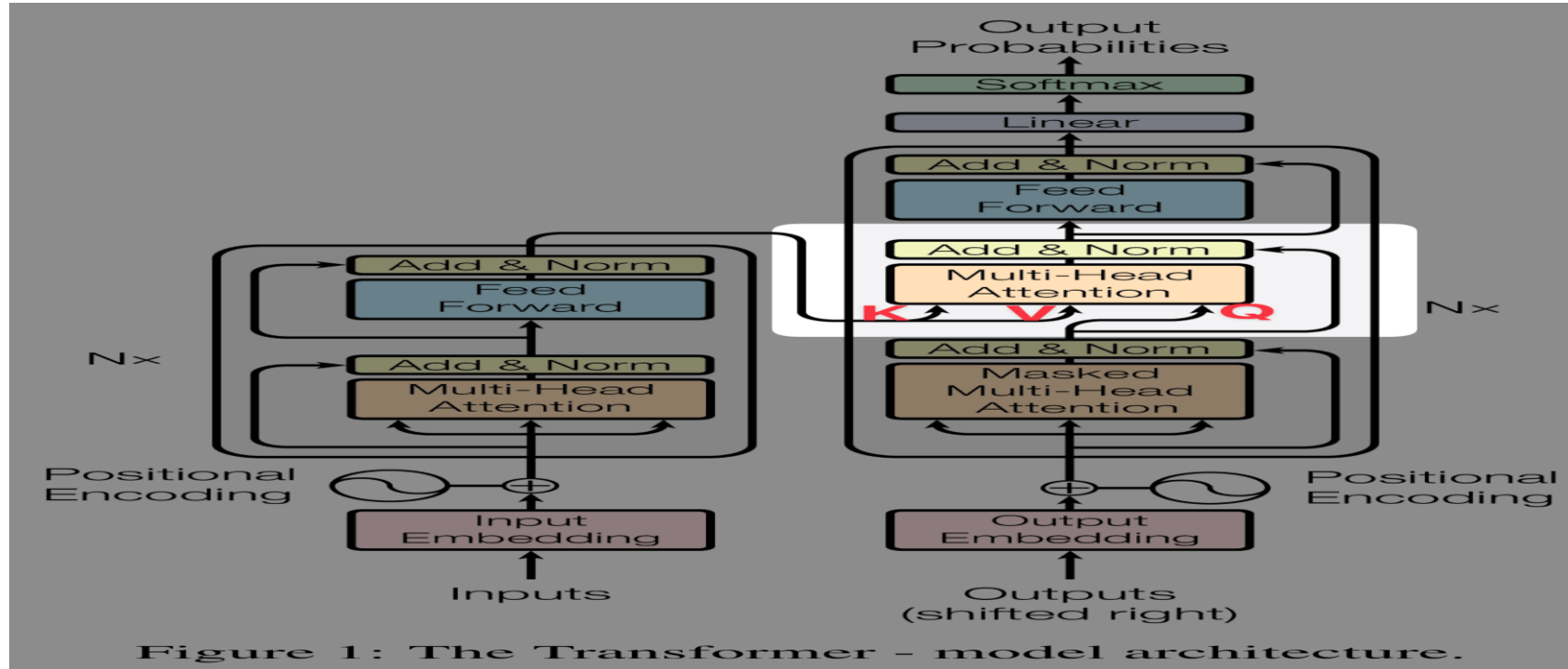


Figure 1: The Transformer - model architecture.

$$\text{CrossAttention}(Q_{dec}, K_{enc}, V_{enc}) = \text{softmax} \left(\frac{Q_{dec} K_{enc}^T}{\sqrt{d_k}} \right) V_{enc}$$



How The Encoder And The Decoder Stack Works

- The word embeddings of the input sequence are passed to the **first encoder**
- These are then transformed and propagated to the next encoder
- **The output from the last encoder in the encoder-stack is passed to all the decoders in the decoder-stack**

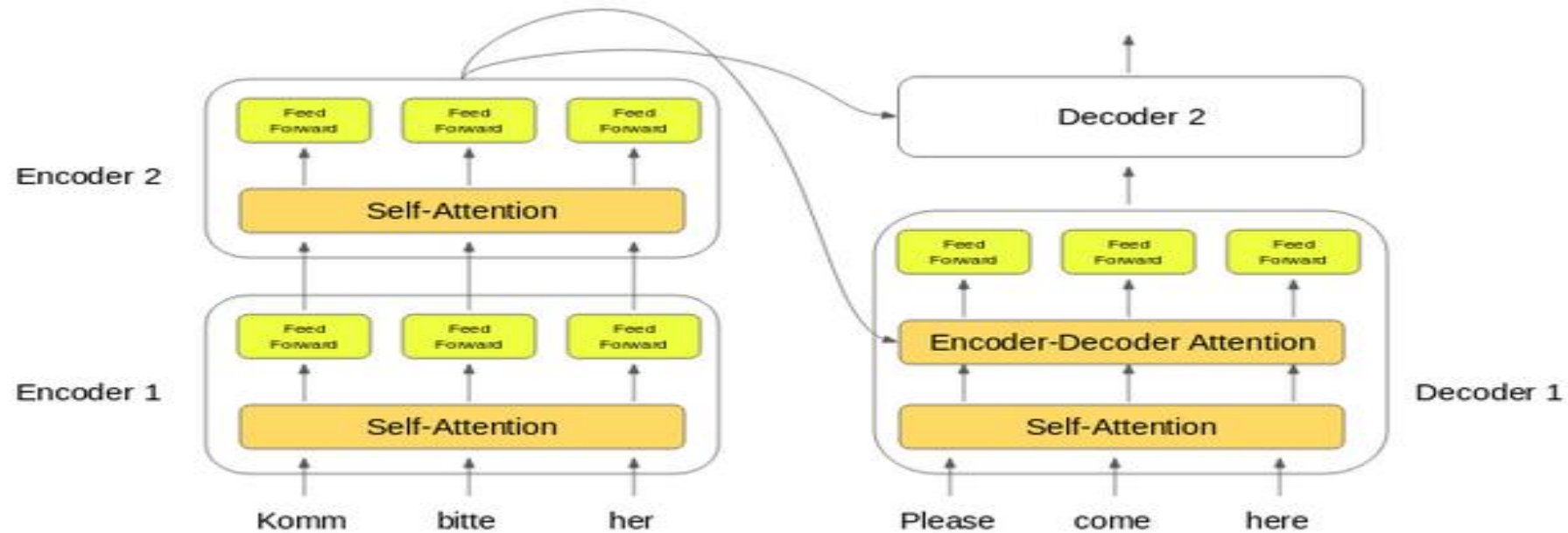


Image source: <https://www.analyticsvidhya.com/blog/2019/06/understanding-transformers-nlp-state-of-the-art-models/>

The Final SoftMax Layer

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)

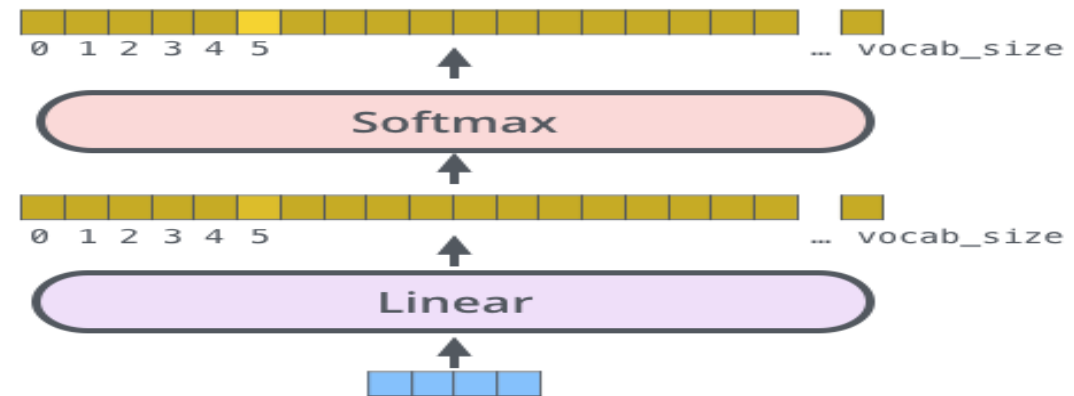
log_probs

logits

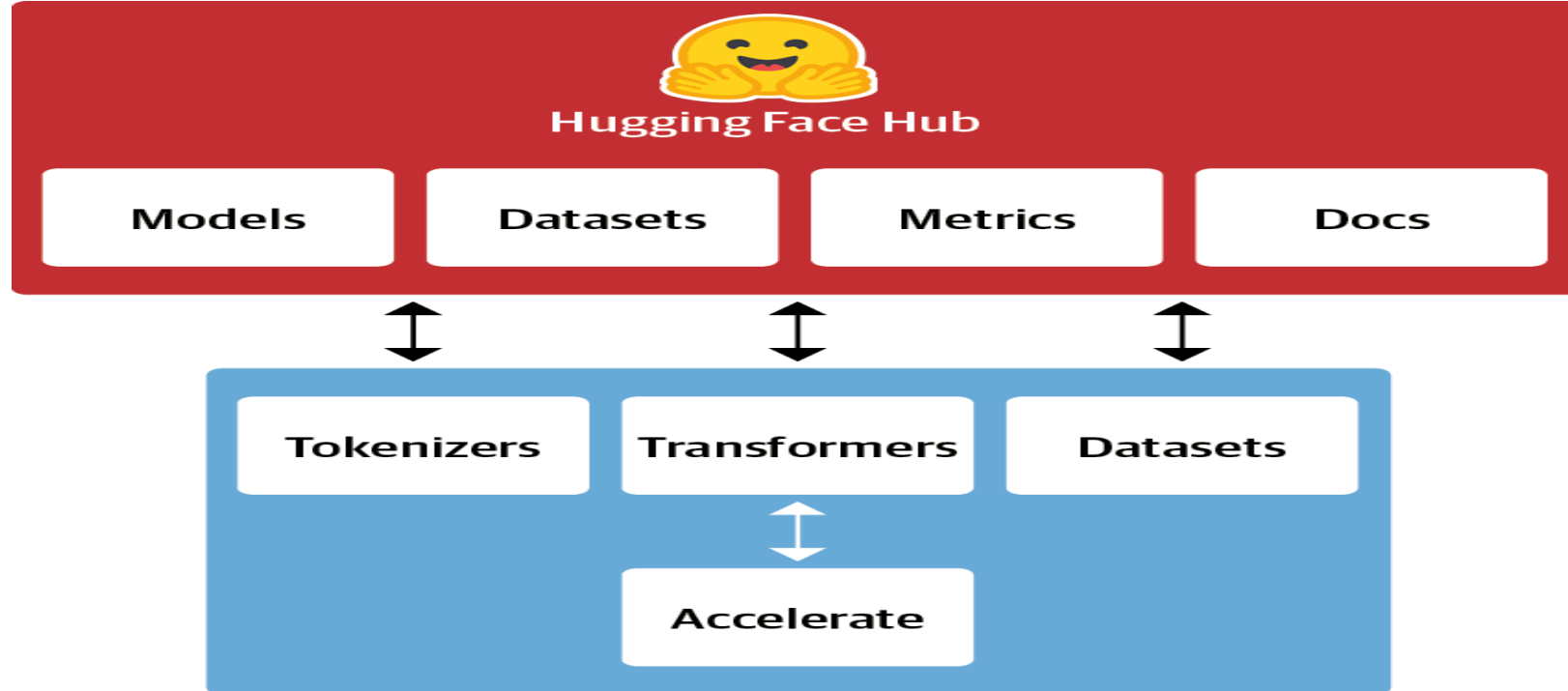
Decoder stack output

am

5



The Hugging Face Ecosystem



The Hugging Face Hub

 **Hugging Face**

[Models](#) [Datasets](#) [Resources](#) [Solutions](#) [Pricing](#)



Tasks

Fill-Mask

Question Answering

Summarization

Table Question Answering

Text Classification

Text Generation

Text2Text Generation

Token Classification

Translation

Zero-Shot Classification

Sentence Similarity

+ 12

Libraries

PyTorch

TensorFlow

JAX

+ 19

Datasets

wikipedia

common_voice

bookcorpus

dcep europarl jrc-acquis

glue

squad

Models

 25,493 Sort: Most Downloads

bert-base-uncased
Fill-Mask • Updated May 18 • 27.5M • ❤️ 42

xlm-roberta-base
Fill-Mask • Updated Sep 16 • 5.88M • ❤️ 9

roberta-large
Fill-Mask • Updated May 21 • 5.26M • ❤️ 15

distilbert-base-uncased
Fill-Mask • Updated Aug 29 • 4.86M • ❤️ 22

gpt2
Text Generation • Updated May 19 • 4.64M • ❤️ 15

Transformer Applications

- Text Classification

```
text = """Dear Amazon, last week I ordered an Optimus Prime action figure  
from your online store in Germany. Unfortunately, when I opened the package,  
I discovered to my horror that I had been sent an action figure of Megatron  
instead! As a lifelong enemy of the Decepticons, I hope you can understand my  
dilemma. To resolve the issue, I demand an exchange of Megatron for the  
Optimus Prime figure I ordered. Enclosed are copies of my records concerning  
this purchase. I expect to hear from you soon. Sincerely, Bumblebee."""
```

```
from transformers import pipeline  
classifier = pipeline("text-classification")  
  
import pandas as pd  
outputs = classifier(text)  
pd.DataFrame(outputs)
```

	label	score
0	NEGATIVE	0.901546



Transformer Applications...

- Named Entity Recognition

```
ner_tagger = pipeline("ner",aggregation_strategy="simple")
outputs = ner_tagger(text)
pd.DataFrame(outputs)
```

	entity_group	score	word	start	end
0	ORG	0.879010	Amazon	5	11
1	MISC	0.990859	Optimus Prime	36	49
2	LOC	0.999755	Germany	90	97
3	MISC	0.556569	Mega	208	212
4	PER	0.590256	##tron	212	216
5	ORG	0.669692	Decept	253	259
6	MISC	0.498350	##icons	259	264
7	MISC	0.775361	Megatron	350	358
8	MISC	0.987854	Optimus Prime	367	380
9	PER	0.812096	Bumblebee	502	511



Transformer Applications...

- Question Answering

```
reader = pipeline("question-answering")  
question = "What does the customer want?"  
outputs = reader(question=question, context=text)  
pd.DataFrame([outputs])
```

	score	start	end	answer
0	0.631291	335	358	an exchange of Megatron



Transformer Applications...

- Summarization

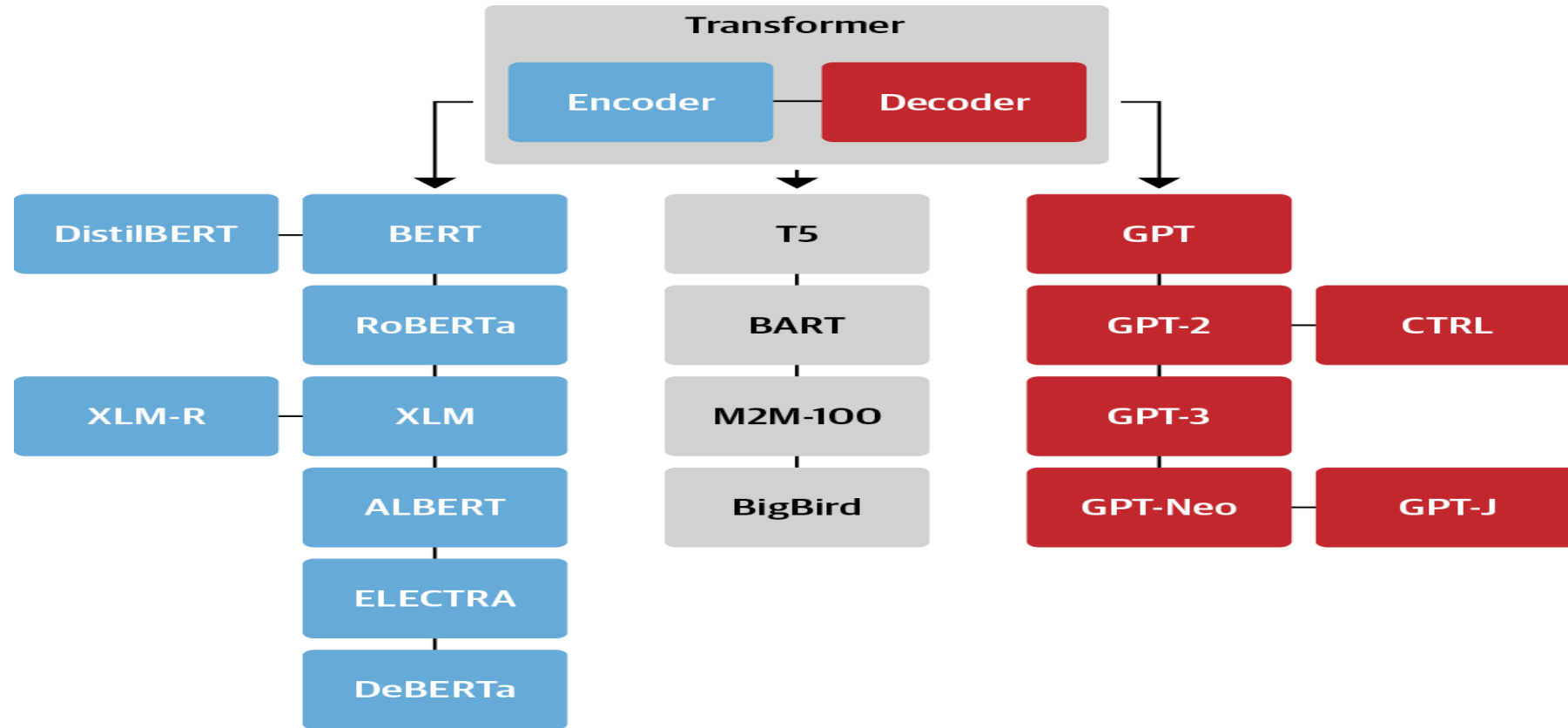
```
text = """Dear Amazon, last week I ordered an Optimus Prime action figure from your online store in Germany. Unfortunately, when I opened the package, I discovered to my horror that I had been sent an action figure of Megatron instead! As a lifelong enemy of the Decepticons, I hope you can understand my dilemma. To resolve the issue, I demand an exchange of Megatron for the Optimus Prime figure I ordered. Enclosed are copies of my records concerning this purchase. I expect to hear from you soon. Sincerely, Bumblebee."""
```

```
summarizer = pipeline("summarization")
outputs = summarizer(text, max_length=80, clean_up_tokenization_spaces=True)
print(outputs[0]['summary_text'])
```

Bumblebee ordered an Optimus Prime action figure from your online store in Germany. Unfortunately, when I opened the package, I discovered to my horror that I had been sent an action figure of Megatron instead.



The Transformer Tree of Life



Challenges with Transformers

- *Language*
- *Data availability*
- *Working with long documents*
- *Transparency*
- *Bias*



Summary



Attention is a mechanism in neural networks that focuses on a specific part of the input and computes its context-dependent summary. It works like a “soft” version of a key-value store.



Self-attention is an attention mechanism that produces the summary of the input by summarizing itself.



The Transformer model applies self-attention repeatedly to gradually transform the input.



Q&A

